机器学习——随机森林+PDP + ICE+ROC +Shap+代价曲线+决策曲线+KS曲线+Lift+六视角横评模型

原创 AI数据之光 Python机器学习ML 2026年10月9日 00:01 重庆

> 感谢关注，一起来学习干货。



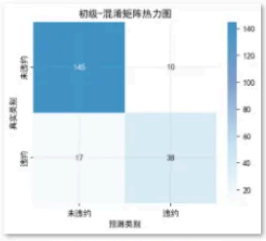
Python机器学习ML
常见数学模型、算法、机器学习、深度学习实战
121篇原创内容

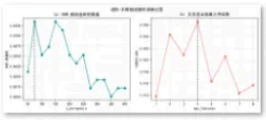
公众号

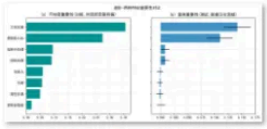
✨ AI工具推荐 ✨

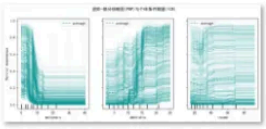
官网：https://artdraw.me/
https://www.ai-api.chat/

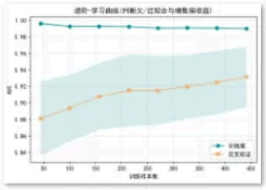
> 随机森林是表格数据上性价比最高的基线之一：不用标准化、几乎不用调参也能跑出不错的AUC。但是真要把一个模型用在业务上，光算准确率远远不够。这篇文章用一份信贷违约数据（`credit` 二分类，8 个特征），从最基础的混淆矩阵一路画到业务决策画布，一共 18 张图，每张图都给出可以直接复制的代码和「怎么读」的分析清单。

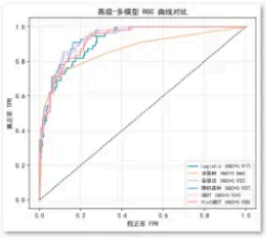

01_基础_混淆矩阵热力图.png

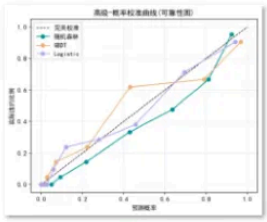

02_进阶_调参曲线.png

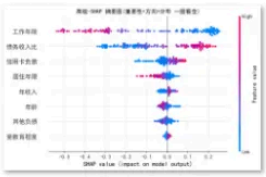

03_进阶_重要性对比.png

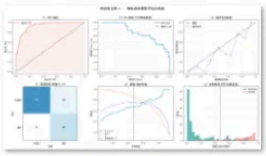

04_进阶_PDP_ICE.png


05_进阶_学习曲线.png

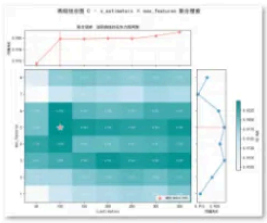

06_高级_多模型ROC对比.png

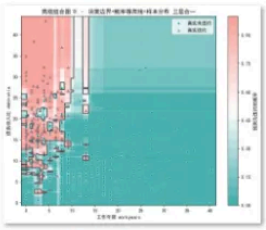

07_高级_概率校准曲线.png


08_高级_SHAP摘要图.png

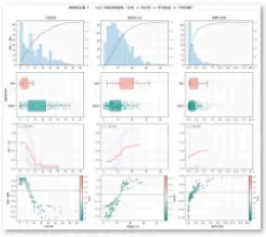

09_高级_评估仪表盘.png

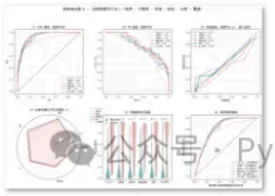

10_高级_重要性三合一.png


11_高级_调参热力图.png


12_高级_决策边界组合图.png









公众号 · Python机器学习ML

0 · 通用准备

把用得到的库、数据、配色一次性配好。后面每张图默认已有 `X_train` / `X_test` / `y_train` / `y_test` / `feature_cols` / `FEATURE_CN` / `CS` / `rf` （已训练好的随机森林）。

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, matplotlib as mpl
from matplotlib.colors import LinearSegmentedColormap
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier

mpl.use('Agg')
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['figure.dpi'] = 110
plt.rcParams['savefig.dpi'] = 150

# 统一配色（整篇文章共用）
CS = {
    'teal': '#009391', 'orange': '#EFB47A', 'purple': '#B3A9ED',
    'bluel': '#C4E3F2', 'pink': '#F1BFDA', 'gray': '#DBDBDB',
    'dark': '#525252', 'red': '#F6847A', 'blue': '#4194C6',
    'mauve': '#CA87A7', 'neg': '#009391', 'pos': '#F6847A',
}
CMAP_RISK = LinearSegmentedColormap.from_list("risk", [CS['teal'], "#FFFFFF", CS['red']])
```

```
CMAP_BLUE = LinearSegmentedColormap.from_list("blues", ["#FFFFFF", CS['bluel'], CS['blue']])
CMAP_SEQ  = LinearSegmentedColormap.from_list("seq",  ["#FFFFFF", CS['bluel'], CS['teal']])

# 数据加载 + 划分
data = pd.read_csv('数据13.1.csv')
feature_cols = [c for c in data.columns if c != 'credit']
FEATURE_CN = {'age':'年龄','education':'受教育程度','workyears':'工作年限',
              'resideyears':'居住年限','income':'年收入','debtratio':'债务收入比',
              'creditdebt':'信用卡负债','otherdebt':'其他负债'}

X, y = data[feature_cols], data['credit']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=10)

# 随机森林（后面所有图都基于它）
rf = RandomForestClassifier(n_estimators=300, max_features='sqrt',
                            class_weight='balanced', oob_score=True,
                            random_state=10, n_jobs=-1).fit(X_train, y_train)
prob_test = rf.predict_proba(X_test)[:, 1]
pred_test = rf.predict(X_test)
```

🎯 **迁移到自己的数据**：只需要改 3 处 —— csv 路径、 `feature_cols` （排除目标列）、目标列名 `credit` 。中文名映射 `FEATURE_CN` 可留可删。

1 · 混淆矩阵热力图

最朴素也最必须的第一张图：模型到底把谁判对了、把谁判错了。四个格子分别对应 TN / FP / FN / TP。

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, pred_test)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap=CMAP_BLUE,
            xticklabels=['未违约','违约'], yticklabels=['未违约','违约'])
plt.xlabel('预测类别'); plt.ylabel('真实类别')
plt.title('混淆矩阵热力图')
plt.savefig('01_基础_混淆矩阵热力图.png', dpi=150, bbox_inches='tight')
```

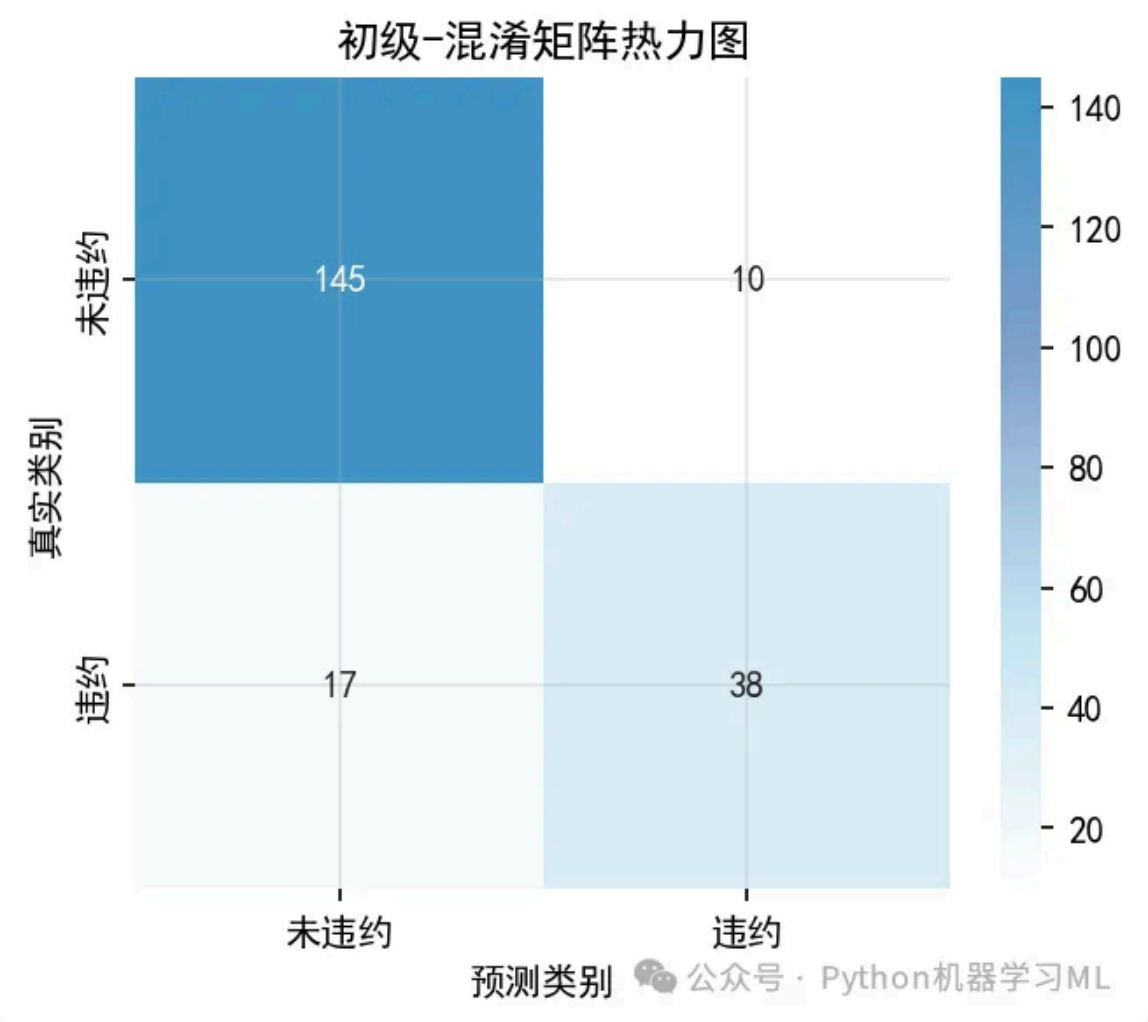


图 1 · 混淆矩阵

怎么读：

- **看对角线深浅**：左上 TN 深 = 正常客户识别到位；右下 TP 深 = 违约客户识别到位。
- **看右上角 FP**：好客户被误拒，拉低通过率，影响营收。
- **看左下角 FN**：违约客户被放过，直接坏账，风控最怕这一格。
- **和总样本量对比**：某一行加起来才是该类的真实数，别只看单格数值。

2 · 调参曲线：OOB + 交叉验证

不用一句一句试参数，让森林自己告诉你 `n_estimators` 、让交叉验证告诉你 `max_features` 。重点是整个过程不碰测试集。

```
from sklearn.model_selection import cross_val_score

n_range = list(range(50, 401, 25))
oob_scores = []
for n in n_range:
```

```
    m = RandomForestClassifier(n_estimators=n, max_features='sqrt',
                               oob_score=True, random_state=10, n_jobs=-1).fit(X_train, y_tra
    oob_scores.append(m.oob_score_)

cv = StratifiedKFold(10, shuffle=True, random_state=10)
feat_range = range(1, X.shape[1]+1)
cv_means = [cross_val_score(RandomForestClassifier(n_estimators=300, max_features=mf,
                            random_state=10, n_jobs=-1),
                            X_train, y_train, cv=cv, scoring='roc_auc').mean()
            for mf in feat_range]

fig, axes = plt.subplots(1, 2, figsize=(12, 4.2))
axes[0].plot(n_range, oob_scores, 'o-', color=CS['teal'])
axes[0].set_xlabel('n_estimators'); axes[0].set_ylabel('OOB 准确率')
axes[0].set_title('(a) OOB 曲线选树的数量')
axes[1].plot(list(feat_range), cv_means, 's-', color=CS['red'])
axes[1].set_xlabel('max_features'); axes[1].set_ylabel('10 折 CV AUC')
axes[1].set_title('(b) 交叉验证选最大特征数')
plt.savefig('02_进阶_调参曲线.png', dpi=150, bbox_inches='tight')
```

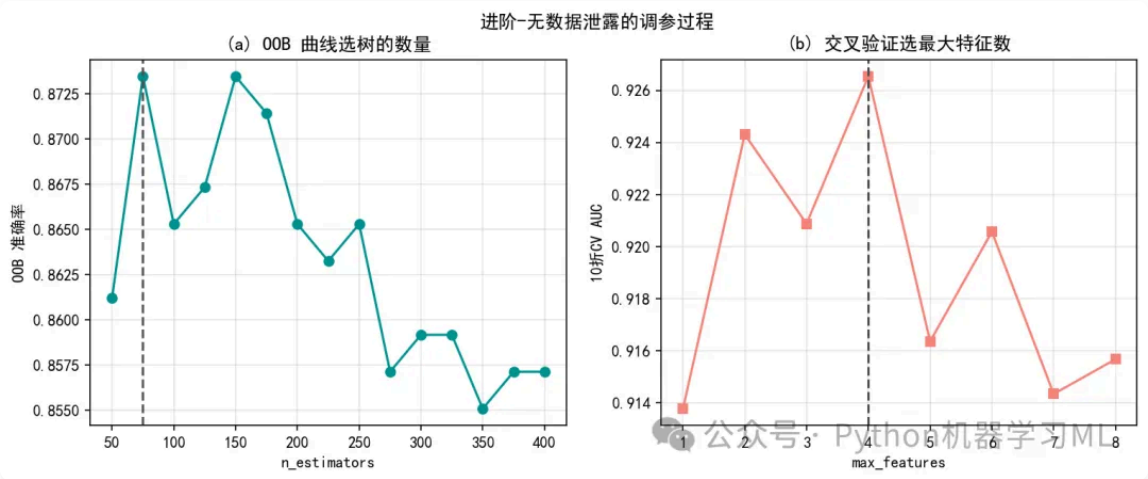


图 2 · OOB 与交叉验证联动调参

怎么读：

- **左图平台期**
  ：OOB 爬升后变平就足够了，再加树只会更慢，不会更准。
- **右图峰值**
  ：AUC 在某个 `max_features` 达到峰值后下降，取峰值即可。
- **曲线抖动**
  ：越抖说明模型越不稳定，考虑把树数再加一点或扩大 CV 折数。
- **不碰测试集**
  ：OOB 和 CV 都是在训练集内部完成的，这是避免数据泄露的关键。

3 · 两种特征重要性对比

不纯度重要性对连续变量和高基数变量有系统性偏好，置换重要性直接衡量"打乱这列 AUC 掉多少"。两者并排一眼就能看出哪些结论可信。

```
from sklearn.inspection import permutation_importance

imp_gini = rf.feature_importances_
perm = permutation_importance(rf, X_test, y_test, n_repeats=20,
                              scoring='roc_auc', random_state=10, n_jobs=-1)
imp_perm = perm.importances_mean

labels = [FEATURE_CN[c] for c in feature_cols]
order = np.argsort(imp_gini); yy = np.arange(len(order))

fig, axes = plt.subplots(1, 2, figsize=(12, 5), sharey=True)
axes[0].barh(yy, imp_gini[order], color=CS['teal'])
axes[0].set_yticks(yy); axes[0].set_yticklabels([labels[i] for i in order])
axes[0].set_title('(a) 不纯度重要性 (训练)')
axes[1].barh(yy, imp_perm[order], xerr=perm.importances_std[order], color=CS['blue'])
axes[1].set_title('(b) 置换重要性 (测试)')
plt.savefig('03_进阶_重要性对比.png', dpi=150, bbox_inches='tight')
```

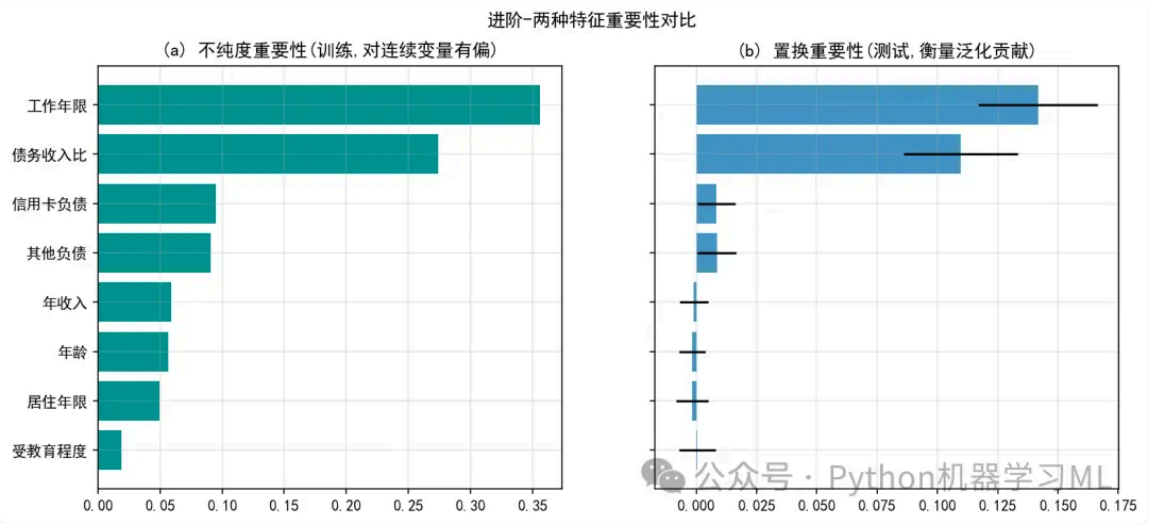


图 3 · 不纯度 vs 置换重要性

怎么读：

- **两图顺序一致**
  的特征才是真关键，比如债务收入比。
- **只在左图突出**
  ：高基数的连续变量常被不纯度法高估，参考置换重要性再决定。
- **误差棒跨 0**
  ：置换重要性置信区间穿过 0 的特征，可以在精简模型时考虑删掉。
- **排序决策**
  ：以置换重要性排序为准，它直接对应"去掉这列能力下降多少"。

4 · PDP + ICE：平均效应与个体轨迹

PDP 告诉你"模型平均怎么用这个特征"，ICE 把每个样本的轨迹叠在一起，暴露"均值掩盖的异质性"。

```
from sklearn.inspection import PartialDependenceDisplay

fig, ax = plt.subplots(figsize=(12, 4.5))
PartialDependenceDisplay.from_estimator(
    rf, X_train, ['workyears', 'debtratio', 'income'],
    kind='both', ax=ax, n_jobs=-1)
plt.suptitle('部分依赖图 (PDP) 与个体条件期望 (ICE)', fontweight='bold')
plt.savefig('04_进阶_PDP_ICE.png', dpi=150, bbox_inches='tight')
```

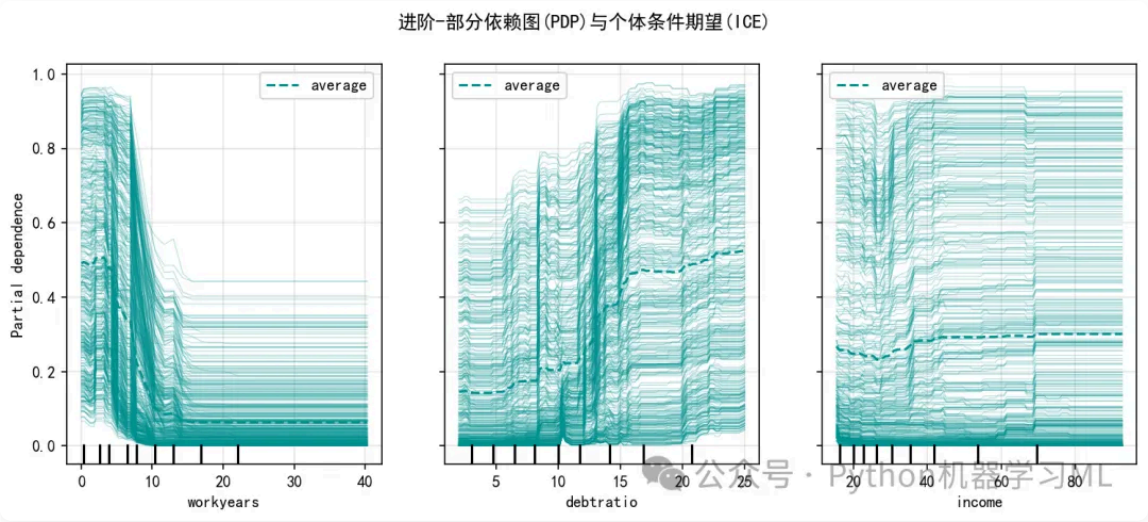


图 4 · 平均效应 + 个体轨迹

怎么读：

- **粗线方向**
  ：单调上升说明"这列越大违约概率越高"，比如债务收入比。
- **细线扇形**
  ：ICE 曲线互相平行说明平均效应可信；散开严重则存在特征交互。
- **平台段**
  ：曲线有拐点就是"分段决策"，可以作为规则抽取的参考。
- **和重要性结合**
  ：重要性高 + PDP 单调的特征，是最适合写进业务策略的。

5 · 学习曲线

它回答一个常见问题：现在的模型是欠拟合还是过拟合，再多搞点数据有没有收益。

```
from sklearn.model_selection import learning_curve

train_sizes, tr, va = learning_curve(
    rf, X_train, y_train, cv=cv, scoring='roc_auc',
    train_sizes=np.linspace(0.1, 1.0, 8), n_jobs=-1)

plt.figure(figsize=(7, 4.5))
plt.plot(train_sizes, tr.mean(1), 'o-', label='训练集', color=CS['teal'])
plt.plot(train_sizes, va.mean(1), 's-', label='交叉验证', color=CS['red'])
plt.fill_between(train_sizes, va.mean(1)-va.std(1), va.mean(1)+va.std(1), alpha=0.15, color=C
```

```
plt.xlabel('训练样本数'); plt.ylabel('AUC'); plt.legend()
plt.title('学习曲线')
plt.savefig('05_进阶_学习曲线.png', dpi=150, bbox_inches='tight')
```

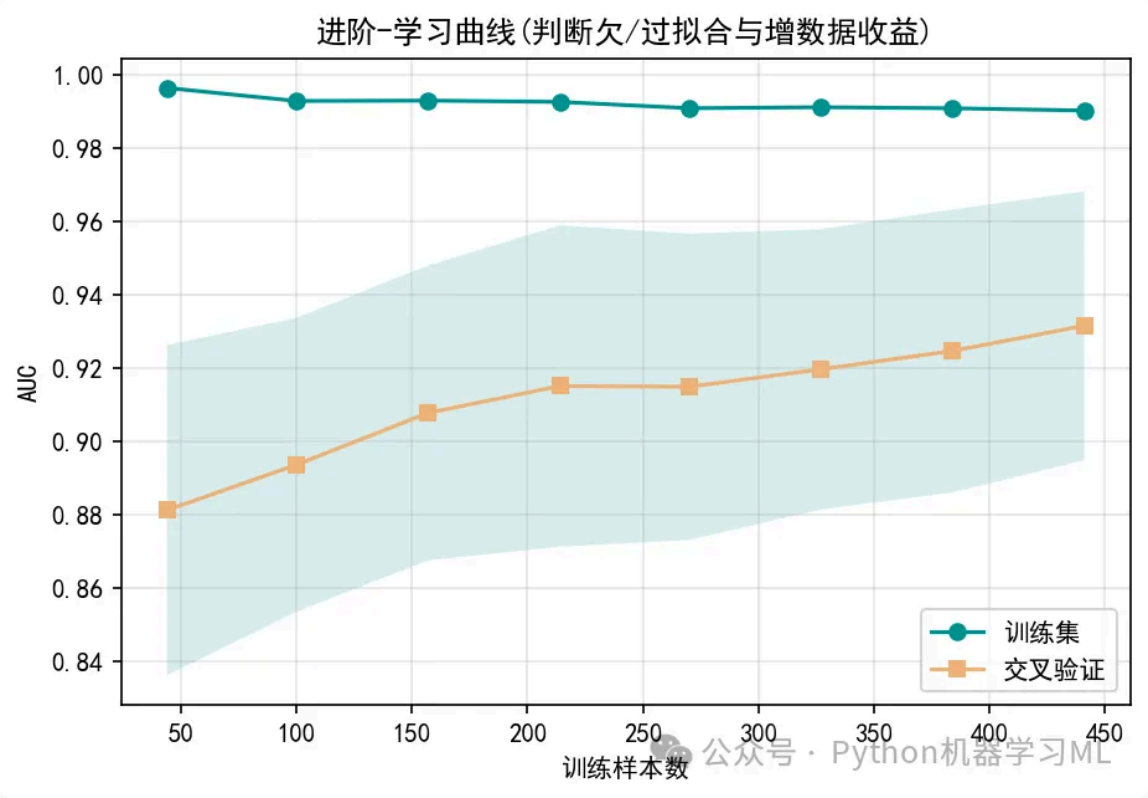


图 5 · 训练 vs 交叉验证 AUC 随样本量变化

怎么读：

- **两条线差距大**：训练远高于验证 → 过拟合，建议加 `min_samples_leaf` 或减深度。
- **两条线都低**：欠拟合，考虑换更强模型或加特征。
- **验证线仍在上升**：加数据还有收益，值得花钱标注。
- **阴影宽窄**：阴影越宽，CV 波动越大，评估结果可信度越低。

6 · 多模型 ROC 曲线

把常见的几个分类器一起拉出来，看谁的 ROC 曲线更靠左上角。

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (BaggingClassifier, GradientBoostingClassifier,
                              HistGradientBoostingClassifier)
from sklearn.metrics import roc_curve, roc_auc_score

models = {
    'Logistic': LogisticRegression(max_iter=1000),
```

```
    '决策树': DecisionTreeClassifier(max_depth=6, random_state=10),
    '装袋法': BaggingClassifier(n_estimators=200, random_state=10, n_jobs=-1),
    '随机森林': rf,
    'GBDT': GradientBoostingClassifier(random_state=10),
    'HistGBDT': HistGradientBoostingClassifier(random_state=10),
}
for name, m in models.items():
    if name != '随机森林':
        m.fit(X_train, y_train)

plt.figure(figsize=(6.5, 5.5))
for name, m in models.items():
    p = m.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, p)
    plt.plot(fpr, tpr, label='%s (AUC=%.3f)' % (name, roc_auc_score(y_test, p)))
plt.plot([0, 1], [0, 1], 'k--', lw=1)
plt.xlabel('假正率 FPR'); plt.ylabel('真正率 TPR')
plt.title('多模型 ROC 曲线对比'); plt.legend(fontsize=8)
plt.savefig('06_高级_多模型ROC对比.png', dpi=150, bbox_inches='tight')
```

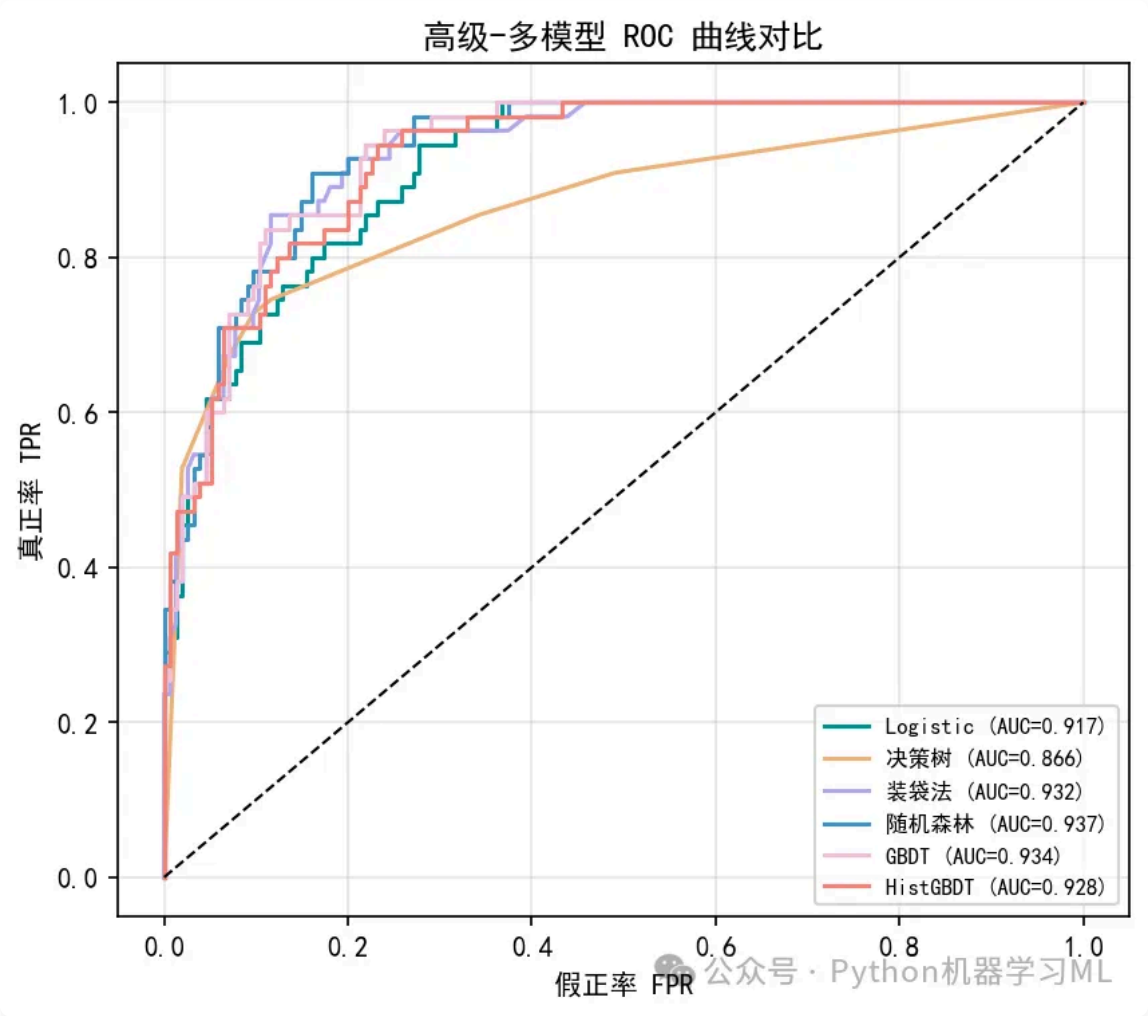


图 6 · 六个模型的 ROC

怎么读：

- **曲线靠左上**
  ：越贴近左上角，排序能力越强。
- **AUC 差距 < 0.01**
  ：性能相当，优先选训练/预测更快、可解释性更好的那个。

- **决策树曲线锯齿**
  ：单模型容易出现"台阶"，提示稳定性差。
- **随机森林 / GBDT 接近**
  ：说明表格数据集上两者可互为替代。

7 · 概率校准曲线

如果输出的概率要用来做阈值切分或者定价，就必须先确认"模型说 0.7 时实际确实接近 70%"。

```
from sklearn.calibration import calibration_curve

fig, ax = plt.subplots(figsize=(6.5, 5))
ax.plot([0, 1], [0, 1], 'k--', lw=1, label='完美校准')
for name in ['随机森林', 'GBDT', 'Logistic']:
    p = models[name].predict_proba(X_test)[:, 1]
    frac_pos, mean_pred = calibration_curve(y_test, p, n_bins=10, strategy='quantile')
    ax.plot(mean_pred, frac_pos, 'o-', label=name)
ax.set_xlabel('预测概率'); ax.set_ylabel('实际违约比例')
ax.set_title('概率校准曲线 (可靠性图)'); ax.legend()
plt.savefig('07_高级_概率校准曲线.png', dpi=150, bbox_inches='tight')
```

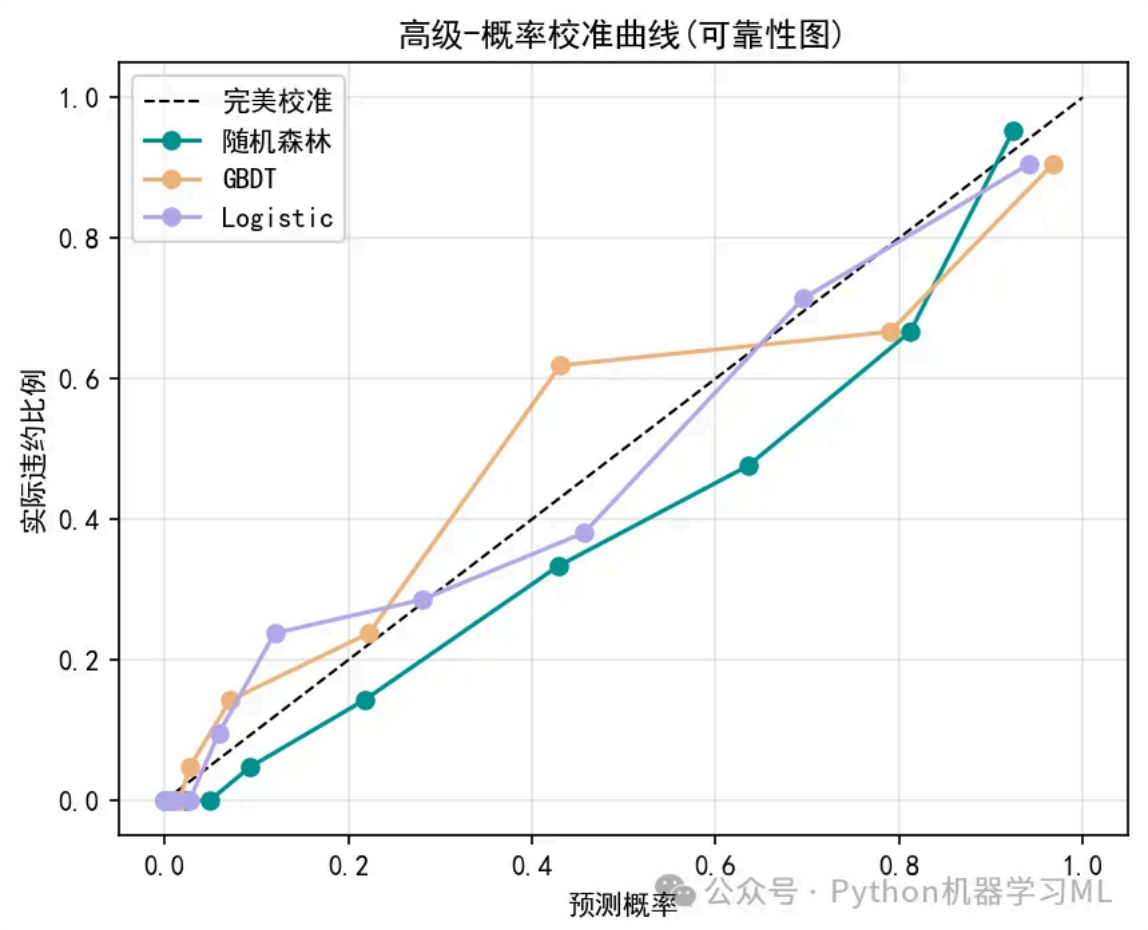


图 7 · 校准曲线

怎么读：

- **贴近对角线**
  ：预测概率可信，可直接用于业务决策。

- **低概率段偏下**
  ：模型高估了低风险人群的违约率，审批侧过严。
- **高概率段偏上**
  ：高风险样本的真实坏账率比预测还高，定价需要调。
- **随机森林微 S 形**
  ：常见现象，可以用 `CalibratedClassifierCV` 做 isotonic 修正。

8 · SHAP 摘要图

SHAP 把"重要性 + 方向 + 分布"三件事画在一张图上，是解释树模型的金标准。

```
import shap

Xs = X_test.sample(min(150, len(X_test)), random_state=10)
explainer = shap.TreeExplainer(rf)
sv = explainer.shap_values(Xs)
sv_pos = sv[1] if isinstance(sv, list) else (sv[..., 1] if sv.ndim == 3 else sv)

plt.figure()
shap.summary_plot(sv_pos, Xs, show=False,
                  feature_names=[FEATURE_CN[c] for c in Xs.columns])
plt.title('SHAP 摘要图')
plt.savefig('08_高级_SHAP摘要图.png', dpi=150, bbox_inches='tight')
```

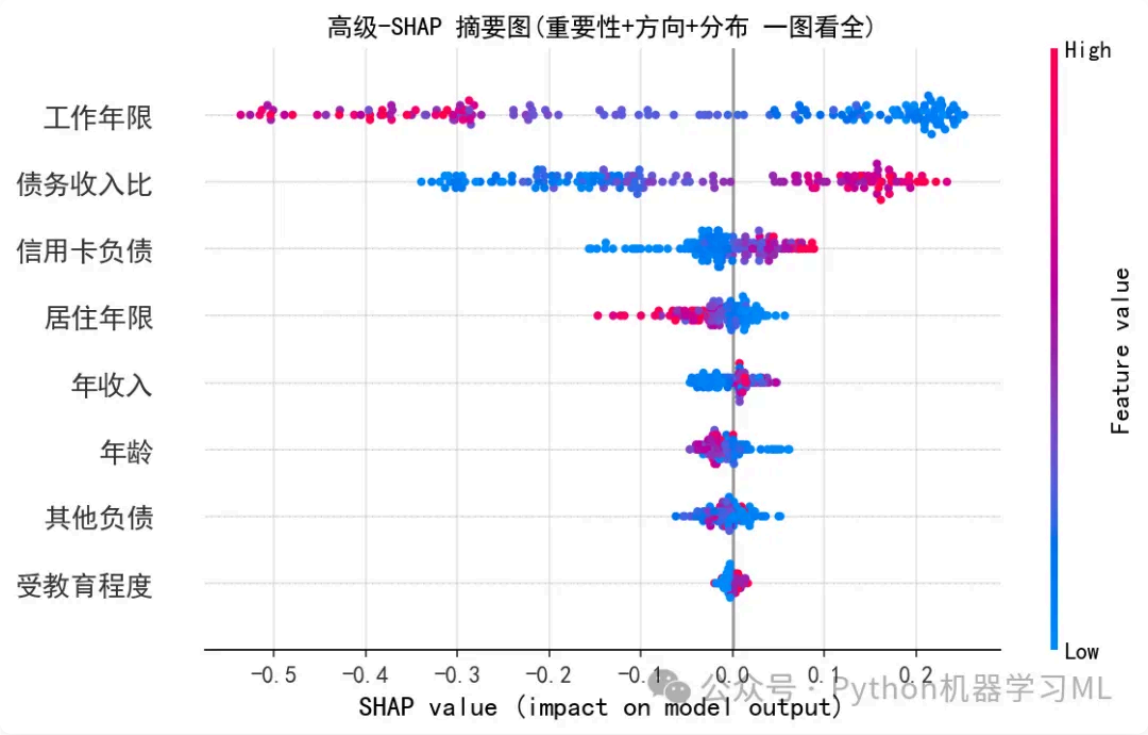


图 8 · SHAP 蜂群图

怎么读：

- **行序 = 重要性**
  ：最上面的特征影响最大。
- **颜色 = 特征值**

：红高蓝低，看颜色和横轴正负的关系能一眼读出方向。

- **横向展开宽**
  ：说明不同人身上这个特征的影响差异大，异质性强。
- **点集中在 0 附近**
  ：这个特征对大多数样本没用，可以考虑精简。

9 · 模型评估仪表盘（2×3）

把评估一个分类器最常用的六张图挤到同一张画布上：ROC、PR、校准、混淆矩阵、阈值-指标、概率分布。一眼就能"从六个视角"审视同一个模型。

```
from sklearn.metrics import (precision_recall_curve, precision_score, recall_score,
                             f1_score, average_precision_score, confusion_matrix)

best_t = 0.4  # 示例：在训练集上选的最优阈值，请换成自己的
fig, axes = plt.subplots(2, 3, figsize=(16, 9))

fpr, tpr, _ = roc_curve(y_test, prob_test)
axes[0,0].plot(fpr, tpr, color=CS['red'], lw=2,
               label='AUC=%.3f' % roc_auc_score(y_test, prob_test))
axes[0,0].fill_between(fpr, tpr, alpha=0.12, color=CS['red'])
axes[0,0].plot([0,1],[0,1],'k--',lw=1); axes[0,0].legend(); axes[0,0].set_title('(1) ROC')

prec, rec, _ = precision_recall_curve(y_test, prob_test)
axes[0,1].plot(rec, prec, color=CS['blue'], lw=2,
               label='AP=%.3f' % average_precision_score(y_test, prob_test))
axes[0,1].axhline(y_test.mean(), ls='--', color=CS['dark']); axes[0,1].legend(); axes[0,1].se

fp, mp = calibration_curve(y_test, prob_test, n_bins=10)
axes[0,2].plot(mp, fp, 'o-', color=CS['purple'])
axes[0,2].plot([0,1],[0,1],'k--',lw=1); axes[0,2].set_title('(3) 校准')

cm = confusion_matrix(y_test, (prob_test >= best_t).astype(int))
axes[1,0].imshow(cm, cmap=CMAP_BLUE)
for i in range(2):
    for j in range(2):
        axes[1,0].text(j, i, cm[i,j], ha='center', va='center', fontsize=13,
                       color='white' if cm[i,j] > cm.max()/2 else 'black')
axes[1,0].set_xticks([0,1]); axes[1,0].set_xticklabels(['未违约','违约'])
axes[1,0].set_yticks([0,1]); axes[1,0].set_yticklabels(['未违约','违约'])
axes[1,0].set_title('(4) 混淆矩阵')

ths = np.linspace(0.05, 0.95, 91)
P = [precision_score(y_test, (prob_test>=t).astype(int), zero_division=0) for t in ths]
R = [recall_score(y_test, (prob_test>=t).astype(int), zero_division=0) for t in ths]
F = [f1_score(y_test, (prob_test>=t).astype(int)) for t in ths]
axes[1,1].plot(ths, P, label='Precision'); axes[1,1].plot(ths, R, label='Recall')
axes[1,1].plot(ths, F, label='F1', lw=2); axes[1,1].axvline(best_t, ls='--', color=CS['dark']
axes[1,1].legend(fontsize=8); axes[1,1].set_title('(5) 阈值-指标')

axes[1,2].hist(prob_test[y_test==0], bins=25, alpha=0.6, label='真实未违约', color=CS['teal']
axes[1,2].hist(prob_test[y_test==1], bins=25, alpha=0.6, label='真实违约', color=CS['red'])
axes[1,2].axvline(best_t, ls='--', color=CS['dark']); axes[1,2].legend(fontsize=8)
```

```
axes[1,2].set_title('(6) 预测概率分布')

plt.suptitle('模型评估仪表盘', fontsize=15, fontweight='bold')
plt.tight_layout(rect=[0,0,1,0.97])
plt.savefig('09_高级_评估仪表盘.png', dpi=150, bbox_inches='tight')
```

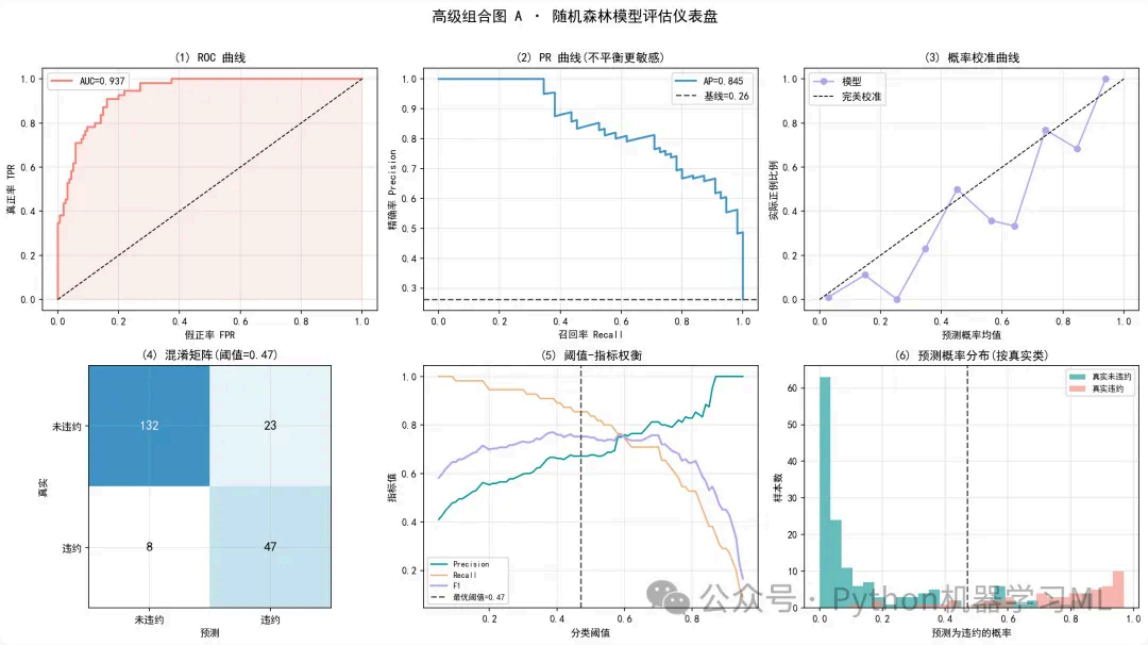


图 9 · 六视角评估仪表盘

怎么读：

- **ROC 与 PR 一起看**
  ：类别不平衡时 PR 更敏感，别只看 ROC。
- **校准弯曲方向**
  ：曲线在对角线上方说明高估风险，在下方说明低估。
- **阈值-指标**
  ：Precision 和 Recall 的交叉点通常接近 F1 最高点，是候选阈值。
- **概率分布**
  ：两个直方图重叠越少，模型越能"把两类分开"，阈值选择的空间越大。

10 · 特征重要性三合一

三种口径（不纯度 / 置换 / SHAP）用同一套特征标签对齐，一眼能看出"哪个结论稳健、哪个只是某种方法的偏好"。

```
shap_mean = np.abs(sv_pos).mean(axis=0)
order = np.argsort(imp_gini); yy = np.arange(len(order))
labels = [FEATURE_CN[c] for c in feature_cols]

fig, axes = plt.subplots(1, 3, figsize=(15.6, 5.2), sharey=True)
axes[0].barh(yy, imp_gini[order], color=CS['teal'])
axes[0].set_yticks(yy); axes[0].set_yticklabels([labels[i] for i in order])
axes[0].set_title('(a) 不纯度重要性')
axes[1].barh(yy, imp_perm[order], xerr=perm.importances_std[order], color=CS['blue'])
```

```
axes[1].set_title('(b) 置换重要性')
axes[2].barh(yy, shap_mean[order], color=CS['purple'])
axes[2].set_title('(c) 平均 |SHAP|')
plt.suptitle('特征重要性三口径对照', fontweight='bold')
plt.tight_layout(rect=[0,0,1,0.95])
plt.savefig('10_高级_重要性三合一.png', dpi=150, bbox_inches='tight')
```

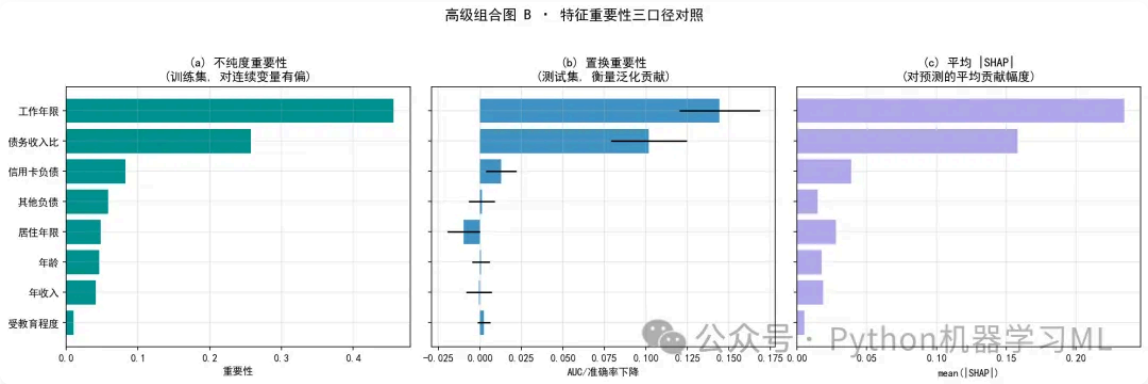


图 10 · 三口径重要性对照

怎么读：

- **三栏都靠前**
  ：铁打的关键特征，报告里重点讲。
- **只在 (a) 高**
  ：不纯度法的偏好，别迷信。
- **(b)(c) 一致、(a) 低**
  ：说明是"实际用得多"的特征，不要被 Gini 挡住。
- **(c) 比 (b) 高**
  ：说明该特征在"个体解释"层面贡献大，但对整体 AUC 贡献小。

11 · 联合调参热力图

一张图把二维联合搜索和两条边际曲线并进来，看清参数的交互效应。

```
cv5 = StratifiedKFold(5, shuffle=True, random_state=10)
n_list, mf_list = list(range(50, 351, 50)), list(range(1, X.shape[1]+1))
grid = np.zeros((len(mf_list), len(n_list)))
for i, mf in enumerate(mf_list):
    for j, n in enumerate(n_list):
        m = RandomForestClassifier(n_estimators=n, max_features=mf,
                                   random_state=10, n_jobs=-1)
        grid[i,j] = cross_val_score(m, X_train, y_train, cv=cv5, scoring='roc_auc').mean()
bi, bj = np.unravel_index(np.argmax(grid), grid.shape)

fig = plt.figure(figsize=(10, 8))
gs = fig.add_gridspec(2, 2, width_ratios=(5, 1.3), height_ratios=(1.3, 5),
                      wspace=0.05, hspace=0.05)
ax_main = fig.add_subplot(gs[1, 0])
ax_top = fig.add_subplot(gs[0, 0], sharex=ax_main)
ax_right = fig.add_subplot(gs[1, 1], sharey=ax_main)
```

```
im = ax_main.imshow(grid, aspect='auto', cmap=CMAP_SEQ, origin='lower')
ax_main.set_xticks(range(len(n_list))); ax_main.set_xticklabels(n_list)
ax_main.set_yticks(range(len(mf_list))); ax_main.set_yticklabels(mf_list)
ax_main.set_xlabel('n_estimators'); ax_main.set_ylabel('max_features')
ax_main.scatter([bj],[bi], marker='*', s=320, color=CS['red'],
                edgecolor='white', zorder=5, label='最优 AUC=%.3f' % grid[bi,bj])
ax_main.legend(loc='lower right', fontsize=8)
for i in range(len(mf_list)):
    for j in range(len(n_list)):
        ax_main.text(j, i, '%.3f' % grid[i,j], ha='center', va='center',
                     color='white', fontsize=7)

ax_top.plot(range(len(n_list)), grid.mean(axis=0), 'o-', color=CS['red'])
ax_top.axvline(bj, ls='--', color=CS['red']); ax_top.set_ylabel('均值AUC')
ax_top.tick_params(labelbottom=False)
ax_right.plot(grid.mean(axis=1), range(len(mf_list)), 's-', color=CS['blue'])
ax_right.axhline(bi, ls='--', color=CS['red']); ax_right.set_xlabel('均值AUC')
ax_right.tick_params(labelleft=False)
fig.colorbar(im, ax=ax_right, fraction=0.08, pad=0.25, label='CV AUC')
plt.suptitle('n_estimators × max_features 联合搜索', fontweight='bold')
plt.savefig('11_高级_调参热力图.png', dpi=150, bbox_inches='tight')
```

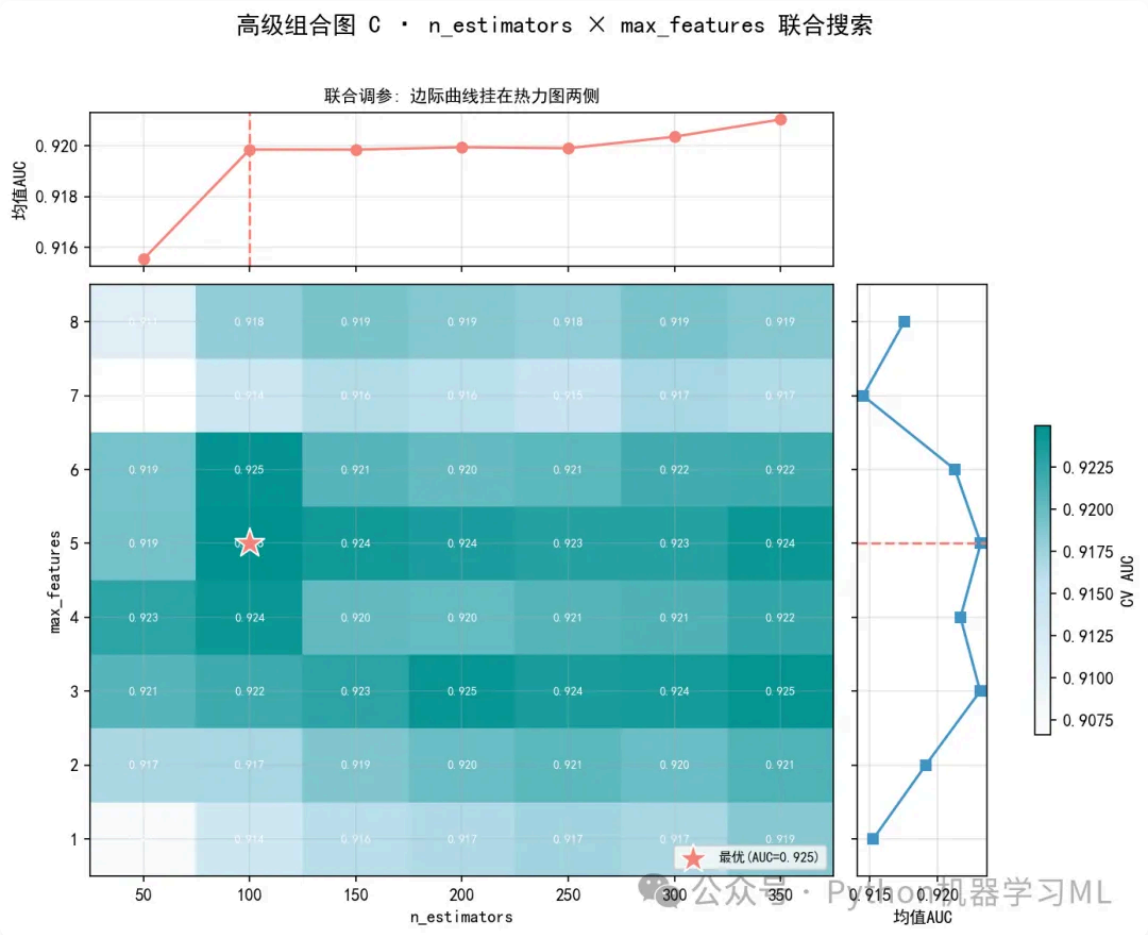


图 11 · 联合调参 + 边际曲线

怎么读：

- **星号位置**
  ：最优格子，优先用它作最终参数。
- **颜色大面积相似**
  ：说明两个超参的性能平台很宽，不用调太细。

- **顶部边际曲线**
  ：n_estimators 对 AUC 影响越小，说明再加树收益越低。
- **右侧边际曲线**
  ：max_features 曲线有明显拐点，拐点附近就是甜区。

12 · 决策边界 + 概率等高线 + 样本

二维投影上同时画三层信息：概率等高线、0.5 决策边界、真实样本点。一张图读完"模型怎么判、人实际在哪"。

```
feats = ['workyears', 'debtratio']
X2 = X[feats].values
Xtr, Xte, ytr, yte = train_test_split(X2, y, test_size=0.3, stratify=y, random_state=10)
mdl = RandomForestClassifier(n_estimators=300, max_features=1,
                             class_weight='balanced', random_state=10, n_jobs=-1).fit(Xtr, yt

x_min, x_max = X2[:,0].min()-1, X2[:,0].max()+1
y_min, y_max = X2[:,1].min()-1, X2[:,1].max()+1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                     np.linspace(y_min, y_max, 300))
prob = mdl.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:,1].reshape(xx.shape)

fig, ax = plt.subplots(figsize=(9, 7))
cf = ax.contourf(xx, yy, prob, levels=20, cmap=CMAP_RISK, alpha=0.75)
ax.contour(xx, yy, prob, levels=[0.5], colors=CS['dark'], linewidths=2)
ax.scatter(X2[y==0,0], X2[y==0,1], c=CS['teal'], s=20, edgecolor='white',
           linewidth=0.5, label='真实未违约')
ax.scatter(X2[y==1,0], X2[y==1,1], c=CS['mauve'], s=22, marker='^',
           edgecolor=CS['dark'], linewidth=0.5, label='真实违约')
ax.set_xlabel('工作年限'); ax.set_ylabel('债务收入比')
ax.set_title('决策边界 + 概率等高线 + 样本分布', fontweight='bold')
ax.legend(loc='upper right')
fig.colorbar(cf, ax=ax, label='预测为违约的概率')
plt.savefig('12_高级_决策边界组合图.png', dpi=150, bbox_inches='tight')
```

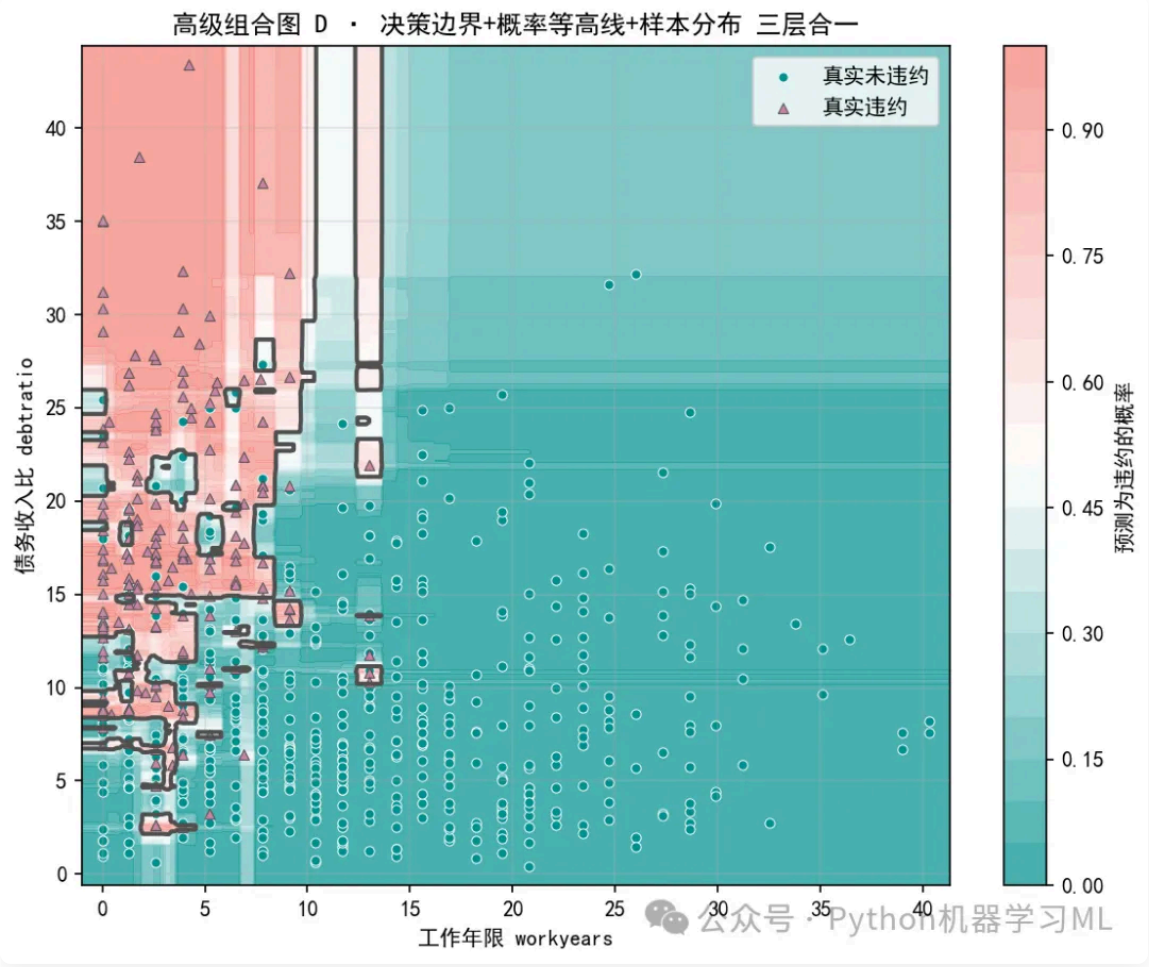


图 12 · 决策边界 + 概率等高线 + 散点

怎么读：

- **深红区**
  ：模型认为违约概率高，看三角点是否真的集中在这里。
- **深色边界线**
  ：0.5 决策线，位置越贴近真实分界，模型越好。
- **绿底里的三角**
  ：被漏判的违约客户，是业务上重点追责的"漏网之鱼"。
- **红底里的圆点**
  ：被错拒的好客户，影响营收。

13 · 数据全景（11 格一图）

EDA 不用点开 11 张图，一张画布看完：类别占比、相关性下三角、分箱违约率、八个特征在两类上的分布差异。

```
fig = plt.figure(figsize=(18, 12))
gs = fig.add_gridspec(3, 4, hspace=0.42, wspace=0.32, height_ratios=(1.25, 1, 1))
cn = [FEATURE_CN[c] for c in feature_cols]

# (a) 类别占比环形
ax = fig.add_subplot(gs[0, 0]); vc = y.value_counts().sort_index()
ax.pie(vc, labels=['未违约 %d' % vc[0], '违约 %d' % vc[1]],
```

```
        colors=[CS['teal'], CS['red']], autopct='%.1f%%', startangle=90,
        pctdistance=0.78, wedgeprops=dict(width=0.42, edgecolor='white'))
ax.text(0, 0, 'N=%d' % len(y), ha='center', va='center', fontsize=13, fontweight='bold')
ax.set_title('(a) 类别占比')

# (b) 相关性下三角
ax = fig.add_subplot(gs[0, 1:3])
corr = X.corr().values; mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
im = ax.imshow(np.ma.masked_where(mask, corr), cmap='RdBu_r', vmin=-1, vmax=1)
for i in range(len(cn)):
    for j in range(i+1):
        ax.text(j, i, '%.2f' % corr[i,j], ha='center', va='center', fontsize=8,
                color='white' if abs(corr[i,j]) > 0.6 else CS['dark'])
ax.set_xticks(range(len(cn))); ax.set_xticklabels(cn, rotation=35, ha='right')
ax.set_yticks(range(len(cn))); ax.set_yticklabels(cn); ax.grid(False)
fig.colorbar(im, ax=ax, fraction=0.035, pad=0.02); ax.set_title('(b) 特征相关性（下三角）')

# (c) 十分位分箱违约率
ax = fig.add_subplot(gs[0, 3])
for col, color in zip(['debtratio','creditdebt','workyears','income'],
                      [CS['red'], CS['orange'], CS['teal'], CS['blue']]):
    bins = pd.qcut(X[col], 10, duplicates='drop')
    rate = y.groupby(bins, observed=True).mean()
    ax.plot(range(1, len(rate)+1), rate.values, 'o-', ms=4, color=color, label=FEATURE_CN[col
ax.axhline(y.mean(), ls='--', color=CS['dark'], lw=1, label='总体违约率')
ax.set_xlabel('特征十分位(低→高)'); ax.set_ylabel('违约率')
ax.set_title('(c) 分箱违约率'); ax.legend(fontsize=7)

# (d)~(k) 八个特征按类别重叠直方图
for i, col in enumerate(feature_cols):
    ax = fig.add_subplot(gs[1 + i//4, i % 4])
    b = np.histogram_bin_edges(X[col], bins=22)
    for cls, c, lab in [(0, CS['teal'], '未违约'), (1, CS['red'], '违约')]:
        v = X.loc[y==cls, col]
        ax.hist(v, bins=b, density=True, alpha=0.55, color=c, label=lab)
        ax.axvline(v.mean(), color=c, ls='--', lw=1.5)
    ax.set_title('(%s) %s' % (chr(ord('d')+i), FEATURE_CN[col]), fontsize=10)
    if i == 0: ax.legend(fontsize=8)

plt.suptitle('数据全景（类别 + 相关性 + 分箱违约率 + 分布差异）',
             fontsize=15, fontweight='bold', y=0.995)
plt.savefig('13_高级_数据全景.png', dpi=150, bbox_inches='tight')
```

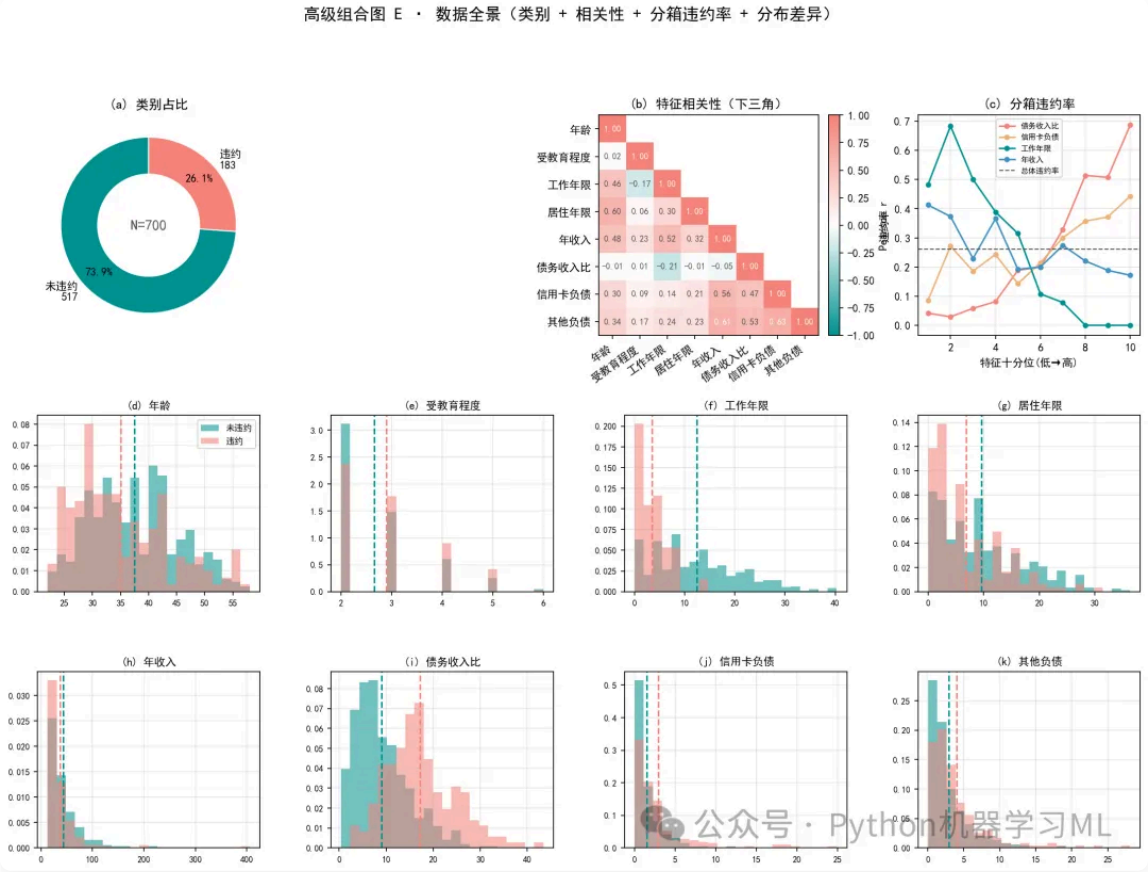


图 13 · 数据全景大图

怎么读：

- **(a) 甜甜圈**
  ：违约类接近 26%，属"轻度不平衡"，不至于必须 SMOTE，但要用 F1/AUC 评估。
- **(b) 下三角**
  ：|r|>0.8 的两格要警觉共线性；对树模型影响不大，对线性模型需要留一删一。
- **(c) 分箱违约率**
  ：曲线单调上升的特征 = 适合写进分数卡；U 形 = 高低两端都危险，需要分段。
- **(d)~(k) 分布**
  ：两条虚线均值离得越远，这个特征越容易判别。

14 · Top3 特征诊断矩阵（4×3）

每一列锁定一个关键特征，从上到下回答四个问题：它长什么样 → 两类能否分开 → 模型平均怎么用它 → 对单个样本贡献多少。PDP、分布、SHAP 第一次在同一 x 轴上对齐。

```
from sklearn.inspection import partial_dependence

top3 = list(np.array(feature_cols)[np.argsort(imp_perm)[::-1][:3]])

fig, axes = plt.subplots(4, 3, figsize=(15, 13.2))
for j, col in enumerate(top3):
    x = X_train[col]
    axes[0,j].hist(x, bins=25, color=CS['bluel'], edgecolor=CS['blue'])
```

```
        axes[0,j].set_title(FEATURE_CN[col], fontsize=12, fontweight='bold')
        ax2 = axes[0,j].twinx(); ax2.grid(False)
        xs = np.sort(x.values)
        ax2.plot(xs, np.arange(1, len(xs)+1)/len(xs), color=CS['dark'], lw=1)

        data_ = [x[y_train==0], x[y_train==1]]
        bp = axes[1,j].boxplot(data_, vert=False, widths=0.5, patch_artist=True, showfliers=False
        for p, c in zip(bp['boxes'], [CS['teal'], CS['red']]):
            p.set_facecolor(c); p.set_alpha(0.5)
        axes[1,j].set_yticks([1,2]); axes[1,j].set_yticklabels(['未违约','违约'])

        pdp = partial_dependence(rf, X_train, [col], kind='both', grid_resolution=40)
        grid_x = pdp['grid_values'][0] if 'grid_values' in pdp else pdp['values'][0]
        ice = pdp['individual'][0]
        pick = np.random.default_rng(10).choice(len(ice), size=min(60, len(ice)), replace=False)
        for line in ice[pick]:
            axes[2,j].plot(grid_x, line, color=CS['purple'], alpha=0.15, lw=0.7)
        axes[2,j].plot(grid_x, pdp['average'][0], color=CS['red'], lw=2.5, label='PDP 平均')
        axes[2,j].set_ylim(0, 1); axes[2,j].legend(fontsize=8, loc='upper left')

        k = list(Xs.columns).index(col)
        others = [c for c in Xs.columns if c != col]
        inter = max(others, key=lambda c: np.nan_to_num(abs(np.corrcoef(Xs[c], sv_pos[:,k])[0,1])
        sc = axes[3,j].scatter(Xs[col], sv_pos[:,k], c=Xs[inter], cmap=CMAP_RISK,
                               s=18, edgecolor=CS['dark'], linewidth=0.3)
        axes[3,j].axhline(0, color=CS['dark'], lw=0.8)
        fig.colorbar(sc, ax=axes[3,j], pad=0.01, fraction=0.05).set_label(FEATURE_CN[inter], font
        axes[3,j].set_xlabel(FEATURE_CN[col])

    for i, label in enumerate(['分布 + 累计', '按类别分布', 'PDP + ICE', 'SHAP 依赖']):
        axes[i,0].set_ylabel(label, fontsize=11, fontweight='bold')
    plt.suptitle('Top3 特征诊断矩阵（分布 → 可分性 → 平均效应 → 个体贡献）',
                 fontsize=14, fontweight='bold')
    plt.tight_layout(rect=[0,0,1,0.97])
    plt.savefig('14_高级_特征诊断矩阵.png', dpi=150, bbox_inches='tight')
```

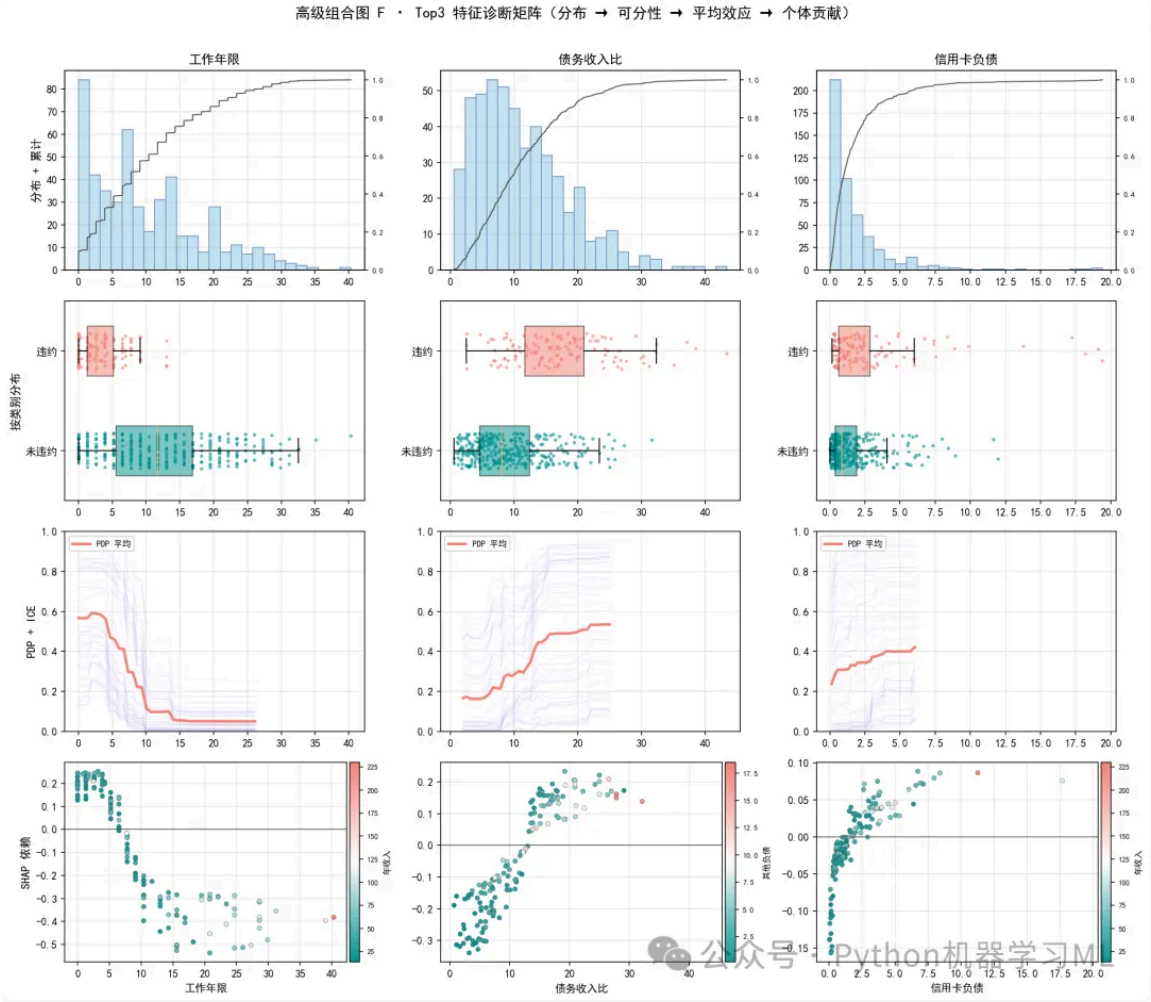


图 14 · Top3 特征四视角对齐

怎么读：

- **行 1 看分布形状**
  ：严重右偏的特征（累计曲线陡起）考虑 log 变换。
- **行 2 看箱线重叠**
  ：两箱错位越明显，这个特征越能把两类分开。
- **行 3 ICE 扇形**
  ：ICE 细线散开严重 = 该特征与其它特征存在交互，PDP 平均会掩盖异质性。
- **行 4 SHAP 散点**
  ：颜色沿纵轴有梯度 → 存在交互效应，颜色所在特征值得深挖。

15 · 多模型横评六合一

六个模型在六个视角下同屏对比，全图共用"模型→颜色"映射。排序、少数类、校准、综合、分离、覆盖，一次性讲完。

```
from sklearn.metrics import roc_curve, precision_recall_curve, roc_auc_score, average_precisi

cmp_rows = []
from sklearn.metrics import f1_score, accuracy_score, cohen_kappa_score
for name, m in models.items():
    p = m.predict_proba(X_test)[:,1]; pr = m.predict(X_test)
```

```
    cmp_rows.append(dict(模型=name, 准确率=accuracy_score(y_test, pr),
                         AUC=roc_auc_score(y_test, p),
                         AP=average_precision_score(y_test, p),
                         F1=f1_score(y_test, pr), Kappa=cohen_kappa_score(y_test, pr)))
cmp_df = pd.DataFrame(cmp_rows).set_index('模型').round(4)

names = list(models)
palette = [CS['blue'], CS['orange'], CS['purple'], CS['red'], CS['teal'], CS['mauve']]
colors = dict(zip(names, palette))
probs = {n: m.predict_proba(X_test)[:,1] for n, m in models.items()}
yv = y_test.values

fig = plt.figure(figsize=(18, 11))
gs = fig.add_gridspec(2, 3, hspace=0.32, wspace=0.28)

ax = fig.add_subplot(gs[0,0])
for n in names:
    fpr, tpr, _ = roc_curve(yv, probs[n])
    ax.plot(fpr, tpr, color=colors[n], lw=2.8 if n=='随机森林' else 1.4,
            label='%s %.3f' % (n, roc_auc_score(yv, probs[n])))
ax.plot([0,1],[0,1],'k--',lw=1); ax.set_title('(1) ROC (AUC)')
ax.legend(fontsize=8, loc='lower right')

ax = fig.add_subplot(gs[0,1])
for n in names:
    prec, rec, _ = precision_recall_curve(yv, probs[n])
    ax.plot(rec, prec, color=colors[n], lw=2.8 if n=='随机森林' else 1.4,
            label='%s %.3f' % (n, average_precision_score(yv, probs[n])))
ax.axhline(yv.mean(), ls='--', color=CS['dark'], lw=1)
ax.set_title('(2) PR (AP)'); ax.legend(fontsize=8)

ax = fig.add_subplot(gs[0,2])
ax.plot([0,1],[0,1],'k--',lw=1)
for n in names:
    fp, mp = calibration_curve(yv, probs[n], n_bins=8, strategy='quantile')
    br = np.mean((probs[n]-yv)**2)
    ax.plot(mp, fp, 'o-', ms=4, color=colors[n], label='%s %.3f' % (n, br))
ax.set_title('(3) 校准 (Brier)'); ax.legend(fontsize=8)

ax = fig.add_subplot(gs[1,0], projection='polar')
metrics = ['准确率','AUC','AP','F1','Kappa']
ang = np.linspace(0, 2*np.pi, len(metrics), endpoint=False).tolist(); ang += ang[:1]
for n in names:
    v = cmp_df.loc[n, metrics].tolist(); v += v[:1]
    ax.plot(ang, v, color=colors[n], lw=2.5 if n=='随机森林' else 1.2, label=n)
    if n == '随机森林': ax.fill(ang, v, color=colors[n], alpha=0.15)
ax.set_xticks(ang[:-1]); ax.set_xticklabels(metrics); ax.set_ylim(0, 1)
ax.set_title('(4) 五指标雷达', pad=16)
ax.legend(fontsize=7, loc='upper right', bbox_to_anchor=(1.35, 1.12))

ax = fig.add_subplot(gs[1,1])
pos = np.arange(len(names))
for cls, off, c in [(0, -0.18, CS['teal']), (1, 0.18, CS['red'])]:
    vp = ax.violinplot([probs[n][yv==cls] for n in names], positions=pos+off,
                       widths=0.34, showmedians=True)
    for b in vp['bodies']:
        b.set_facecolor(c); b.set_alpha(0.55); b.set_edgecolor(CS['dark'])
    for k in ('cbars','cmins','cmaxes','cmedians'):
```

```
        vp[k].set_color(CS['dark']); vp[k].set_linewidth(0.8)
ax.set_xticks(pos); ax.set_xticklabels(names, fontsize=9)
ax.set_ylim(-0.02, 1.12); ax.set_ylabel('预测违约概率')
ax.set_title('(5) 两类概率分离度')

ax = fig.add_subplot(gs[1,2])
xs = np.arange(1, len(yv)+1)/len(yv)
for n in names:
    cum = np.cumsum(yv[np.argsort(-probs[n])])/yv.sum()
    ax.plot(xs, cum, color=colors[n], lw=2.8 if n=='随机森林' else 1.4,
            label='%s 前20%%捕获%.0f%%' % (n, cum[int(0.2*len(yv))-1]*100))
ax.plot([0,1],[0,1],'k--',lw=1); ax.axvline(0.2, color=CS['gray'], lw=1)
ax.set_title('(6) 累积捕获曲线'); ax.legend(fontsize=8)

plt.suptitle('多模型横评六合一', fontsize=15, fontweight='bold')
plt.savefig('15_高级_多模型横评六合一.png', dpi=150, bbox_inches='tight')
```

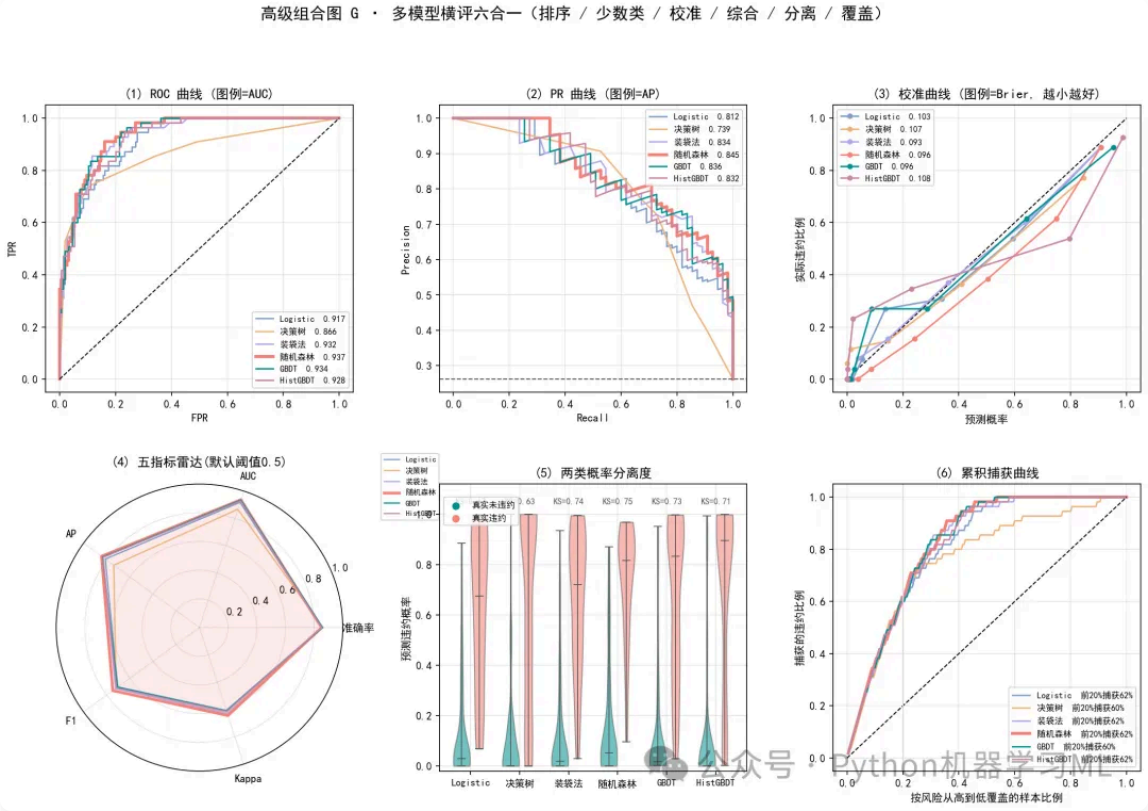


图 15 · 六视角横评六个模型

怎么读：

- **(1)(2) 一起看**
  ：ROC 相近但 PR 分化大 → 不平衡场景下模型差异真实存在。
- **(3) Brier**
  ：越小越好；校准曲线弯曲 + Brier 高 = 概率不可直接用。
- **(4) 雷达**
  ：面积最大的那条是综合最优，但要同时看默认阈值是否公平。
- **(5) 小提琴**
  ：两个颜色的小提琴重叠面积越小，这个模型"排序能力"越强。
- **(6) 累积捕获**

：按风险高到低覆盖 20% 客户能抓多少违约，这是业务侧最关心的 Lift。

16 · SHAP 深度解析（全局 + 两个典型个体）

左上蜂群看全局方向，右上把正/负贡献拆开，下排挑最"确定违约"和最"确定不违约"的两个样本各画一张瀑布图。整体与个体同画布对读。

```
mean_abs = np.abs(sv_pos).mean(0)
order = np.argsort(mean_abs)
labels_s = [FEATURE_CN[c] for c in Xs.columns]
base = float(np.atleast_1d(explainer.expected_value)[-1])

fig = plt.figure(figsize=(17, 11))
gs = fig.add_gridspec(2, 2, hspace=0.3, wspace=0.42, width_ratios=(1.25, 1))

ax = fig.add_subplot(gs[0,0])
rng = np.random.default_rng(10)
for row, k in enumerate(order):
    v = Xs.iloc[:,k].values; vn = (v-v.min())/(np.ptp(v)+1e-12)
    jitter = rng.normal(0, 0.09, len(v))
    sc = ax.scatter(sv_pos[:,k], row+jitter, c=vn, cmap=CMAP_RISK, s=10, alpha=0.85)
ax.axvline(0, color=CS['dark'], lw=0.8)
ax.set_yticks(range(len(order))); ax.set_yticklabels([labels_s[k] for k in order])
cb = fig.colorbar(sc, ax=ax, pad=0.01, fraction=0.04)
cb.set_ticks([0,1]); cb.set_ticklabels(['低','高']); cb.set_label('特征值')
ax.set_xlabel('SHAP 值'); ax.set_title('(a) 蜂群图')

ax = fig.add_subplot(gs[0,1])
pos_c = np.where(sv_pos>0, sv_pos, 0).mean(0)
neg_c = np.where(sv_pos<0, sv_pos, 0).mean(0)
yy = np.arange(len(order))
ax.barh(yy, pos_c[order], color=CS['red'], label='推高违约(+)')
ax.barh(yy, neg_c[order], color=CS['teal'], label='拉低违约(-)')
ax.scatter(mean_abs[order], yy, marker='D', color=CS['dark'], s=28, zorder=5, label='mean|SHA
ax.axvline(0, color=CS['dark'], lw=0.8)
ax.set_yticks(yy); ax.set_yticklabels([labels_s[k] for k in order])
ax.legend(fontsize=8); ax.set_title('(b) 正向 vs 负向贡献')

def _waterfall(ax, sv_row, x_row, base, title, max_show=8):
    ord_ = np.argsort(np.abs(sv_row))[::-1][:max_show][::-1]
    cur = base + sv_row.sum() - sv_row[ord_].sum()
    for row, k in enumerate(ord_):
        v = sv_row[k]
        ax.barh(row, v, left=cur, color=CS['red'] if v>0 else CS['teal'], edgecolor='white',
        ax.text(cur+v+(0.004 if v>0 else -0.004), row, '%+.3f' % v,
                va='center', ha='left' if v>0 else 'right', fontsize=8)
        cur += v
    ax.axvline(base, color=CS['dark'], ls='--', lw=1); ax.axvline(cur, color=CS['red'], lw=1.
    ax.set_yticks(range(len(ord_)))
    ax.set_yticklabels(['%s = %g' % (labels_s[k], round(x_row[k], 2)) for k in ord_], fontsiz
    ax.set_xlabel('违约概率 (虚线=基准 %.3f, 红线=预测 %.3f)' % (base, cur))
    ax.set_title(title)

pred_s = base + sv_pos.sum(1)
i_hi, i_lo = int(np.argmax(pred_s)), int(np.argmin(pred_s))
```

```
for slot, idx, tag in [(gs[1,0], i_hi, '最高风险样本'), (gs[1,1], i_lo, '最低风险样本')]:
    _waterfall(fig.add_subplot(slot), sv_pos[idx], Xs.iloc[idx].values, base, '(%s) %s' %
               ('c' if idx==i_hi else 'd', tag))

plt.suptitle('SHAP 深度解析', fontsize=15, fontweight='bold')
plt.savefig('16_高级_SHAP深度解析.png', dpi=150, bbox_inches='tight')
```

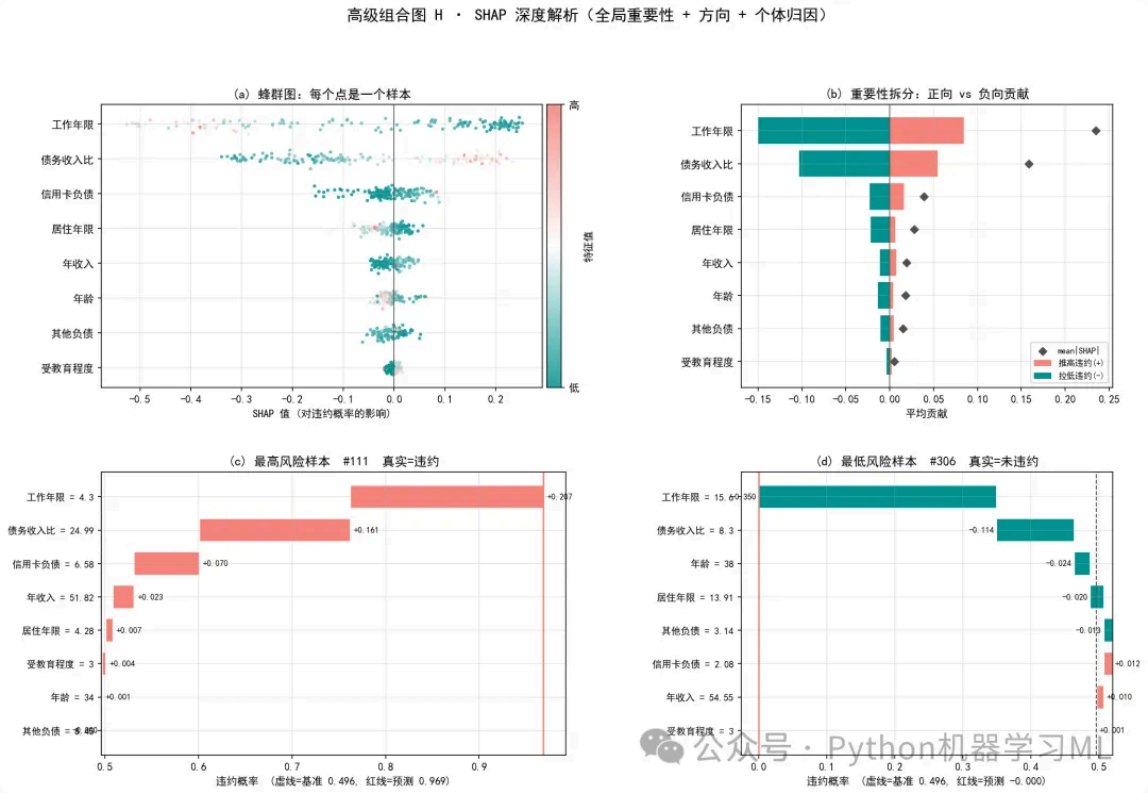


图 16 · 全局 + 个体 SHAP 对读

怎么读：

- **(a) 蜂群**
  ：行序给重要性，颜色给方向。某行颜色左低右高 = "特征值越大越推高违约"。
- **(b) 正负拆分**
  ：红条远长于蓝条 = 该特征几乎只会推高风险，是单向风险因子。
- **(c) 高风险瀑布**
  ：从基准概率出发每一步的贡献，可以直接写进拒绝客户的"理由说明"。
- **(d) 低风险瀑布**
  ：蓝条叠加说明这个客户在多个维度都被模型信任。

17 · 业务决策画布

这张图回答："模型分数怎么变成业务决策"。阈值在训练集 5 折交叉的 OOF 概率上选（不碰测试集），在测试集上展示效果。代价假设漏判违约 FN=5，误拒好客户 FP=1，按业务自己改。

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import f1_score, confusion_matrix
```

```
COST_FN, COST_FP = 5.0, 1.0
cv5 = StratifiedKFold(5, shuffle=True, random_state=10)
oof = cross_val_predict(rf, X_train, y_train, cv=cv5, method='predict_proba', n_jobs=-1)[:,1]
yv, prob = y_test.values, prob_test
ths = np.linspace(0.02, 0.98, 97)

def cost_curve(y_, p_):
    return np.array([(COST_FN*((p_=t)&(y_==0)).sum())/len(y_) for t in ths])
cost_oof, cost_te = cost_curve(y_train.values, oof), cost_curve(yv, prob)
t_cost = ths[int(np.argmin(cost_oof))]
t_f1 = ths[int(np.argmax([f1_score(y_train, (oof>=t).astype(int)) for t in ths]))]

fig = plt.figure(figsize=(18, 11))
gs = fig.add_gridspec(2, 3, hspace=0.33, wspace=0.28)

ax = fig.add_subplot(gs[0,0])
ax.plot(ths, cost_oof, color=CS['blue'], lw=2, label='训练 OOF(用于选阈值)')
ax.plot(ths, cost_te, color=CS['red'], lw=1.5, ls='-.', label='测试集(仅展示)')
ax.axvline(t_cost, color=CS['dark'], ls='--', label='代价最优 %.2f' % t_cost)
ax.axvline(t_f1, color=CS['orange'], ls=':', lw=2, label='F1 最优 %.2f' % t_f1)
ax.axvline(0.5, color=CS['gray'], lw=1.5, label='默认 0.50')
ax.set_xlabel('分类阈值'); ax.set_ylabel('人均期望代价')
ax.set_title('(1) 期望代价曲线 (FN=%g, FP=%g)' % (COST_FN, COST_FP)); ax.legend(fontsize=8)

sub = gs[0,1].subgridspec(1, 3, wspace=0.08)
for k, (t, nm) in enumerate([(0.5, '默认'), (t_f1, 'F1最优'), (t_cost, '代价最优')]):
    a = fig.add_subplot(sub[0, k])
    cm = confusion_matrix(yv, (prob>=t).astype(int))
    a.imshow(cm, cmap=CMAP_BLUE); a.grid(False)
    for i in range(2):
        for j in range(2):
            a.text(j, i, cm[i,j], ha='center', va='center', fontsize=12,
                   color='white' if cm[i,j]>cm.max()/2 else 'black')
    c = (COST_FN*cm[1,0] + COST_FP*cm[0,1])/cm.sum()
    a.set_title('%s t=%.2f\n代价=%.3f' % (nm, t, c), fontsize=9)
    a.set_xticks([0,1]); a.set_xticklabels(['预测0','预测1'], fontsize=8)
    a.set_yticks([0,1]); a.set_yticklabels(['真0','真1'], fontsize=8)
    if k > 0: a.tick_params(labelleft=False)
fig.text(0.5, 0.945, '(2) 三种阈值下的测试集混淆矩阵', ha='center', fontsize=12)

ax = fig.add_subplot(gs[0,2])
ps = np.sort(np.unique(prob))
cdf_bad = [(prob[yv==1]<=s).mean() for s in ps]
cdf_good = [(prob[yv==0]<=s).mean() for s in ps]
diff = np.array(cdf_good)-np.array(cdf_bad); ki = int(np.argmax(diff))
ax.plot(ps, cdf_good, color=CS['teal'], lw=2, label='未违约 CDF')
ax.plot(ps, cdf_bad, color=CS['red'], lw=2, label='违约 CDF')
ax.vlines(ps[ki], cdf_bad[ki], cdf_good[ki], color=CS['dark'], lw=2.5)
ax.text(ps[ki]+0.02, (cdf_bad[ki]+cdf_good[ki])/2,
        'KS=%.3f\n@%.2f' % (diff[ki], ps[ki]), fontsize=10, fontweight='bold')
ax.set_xlabel('预测违约概率'); ax.set_ylabel('累计比例')
ax.set_title('(3) KS 曲线'); ax.legend(fontsize=8)

ax = fig.add_subplot(gs[1, 0:2])
dec = 9 - pd.qcut(pd.Series(prob).rank(method='first'), 10, labels=False)
tab = pd.DataFrame({'d': dec, 'y': yv, 'p': prob}).groupby('d').agg(
    n=('y','size'), bad=('y','sum'), rate=('y','mean'), pm=('p','mean'))
tab['lift'] = tab['rate']/yv.mean()
```

```
tab['cum_capture'] = tab['bad'].cumsum()/yv.sum()
xl = np.arange(10)
ax.bar(xl-0.2, tab['n']-tab['bad'], 0.4, color=CS['teal'], alpha=0.7, label='未违约人数')
ax.bar(xl+0.2, tab['bad'], 0.4, color=CS['red'], alpha=0.85, label='违约人数')
ax.set_xticks(xl); ax.set_xticklabels(['D%d' % (i+1) for i in xl])
ax.set_xlabel('风险十分位 (D1=最高风险)'); ax.set_ylabel('人数')
ax2 = ax.twinx(); ax2.grid(False)
ax2.plot(xl, tab['rate'], 'o-', color=CS['red'], lw=2, label='实际违约率')
ax2.plot(xl, tab['pm'], 's--', color=CS['purple'], lw=1.5, label='平均预测概率')
ax2.plot(xl, tab['cum_capture'], '^-', color=CS['blue'], lw=1.5, label='累计捕获率')
ax2.set_ylim(0, 1.15); ax2.set_ylabel('比例')
h1,l1 = ax.get_legend_handles_labels(); h2,l2 = ax2.get_legend_handles_labels()
ax.legend(h1+h2, l1+l2, fontsize=8, ncol=5, loc='upper center')
ax.set_title('(4) 风险十分位分层表')

ax = fig.add_subplot(gs[1,2])
approve = np.array([(prob<t).mean() for="" t="" in="" ths])="" badrate="np.array([yv[prob<t].
```

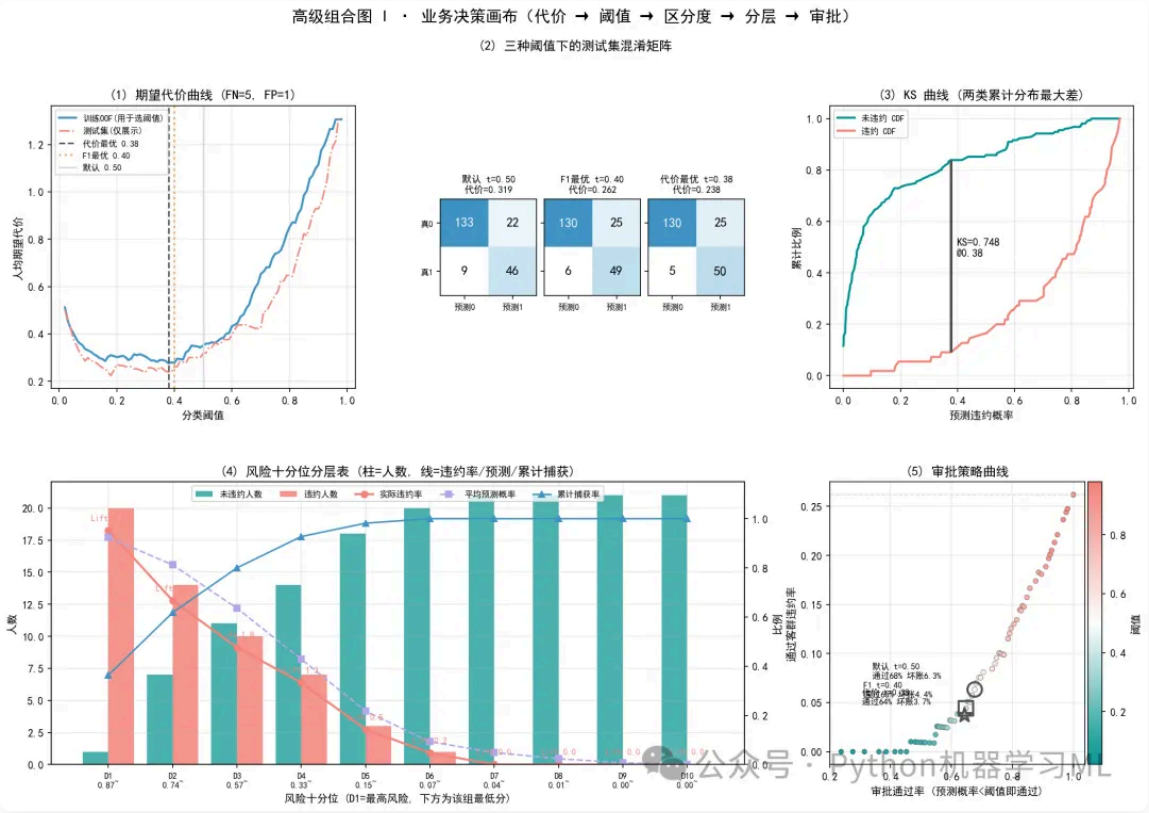


图 17 · 从模型分数到审批策略

怎么读：

- **(1) 代价曲线**
  ：OOF 和测试曲线趋势一致说明阈值选对了，谷底就是建议阈值。
- **(2) 三联混淆矩阵**
  ：对比"默认 / F1 / 代价"三套阈值，右边两套通常能把 FN 大幅压下来。
- **(3) KS 曲线**
  ：两类累计分布最大差距，KS>0.3 风控可用，>0.4 很好。
- **(4) 分层表**
  ：D1 组违约率应该是均值的几倍（Lift），业务可以直接拿这列设计分层策略。
- **(5) 审批曲线**
  ：横轴通过率 vs 纵轴通过客群违约率，划一条可接受坏账率就对应了通过率。

18 · 误分类诊断

按预测结果把测试集切成 TP/FN/FP/TN，看模型在哪儿错、错的人像谁。FN（漏判违约）常常"长得像好客户"，这张图能直观看出来。

```
t = t_cost  # 用业务画布选出来的阈值
pred = (prob>=t).astype(int)
grp = np.select([(yv==1)&(pred==1),(yv==1)&(pred==0),(yv==0)&(pred==1)],
                ['TP','FN','FP'], 'TN')
G = ['TP','FN','FP','TN']
GCN = {'TP':'TP 正确识别违约','FN':'FN 漏判违约','FP':'FP 误拒好客户','TN':'TN 正确放行'}
GC = {'TP':CS['red'],'FN':CS['orange'],'FP':CS['purple'],'TN':CS['teal']}
GM = {'TP':'^','FN':'X','FP':'D','TN':'o'}
labels = [FEATURE_CN[c] for c in X_test.columns]
Z = (X_test - X_test.mean())/X_test.std()
zmean = pd.DataFrame({g: Z[grp==g].mean() for g in G}).T

fig = plt.figure(figsize=(18, 11))
gs = fig.add_gridspec(2, 3, hspace=0.32, wspace=0.3)

ax = fig.add_subplot(gs[0,0])
cnt = [int((grp==g).sum()) for g in G]
bars = ax.bar(G, cnt, color=[GC[g] for g in G], edgecolor=CS['dark'])
for b, g, c in zip(bars, G, cnt):
    pm = prob[grp==g].mean() if c else np.nan
    ax.text(b.get_x()+b.get_width()/2, c+1, '%d\n均概率%.2f' % (c, pm), ha='center', fontsize
ax.set_ylim(0, max(cnt)*1.25); ax.set_ylabel('样本数')
ax.set_title('(a) 四组规模 (阈值=%.2f)' % t)

ax = fig.add_subplot(gs[0,1])
fx, fy = 'debtratio', 'workyears'
for g in ['TN','TP','FP','FN']:
    m = grp==g
    ax.scatter(X_test.loc[m, fx], X_test.loc[m, fy], marker=GM[g], color=GC[g],
               s=30+70*prob[m], edgecolor=CS['dark'], linewidth=0.4,
               alpha=0.9 if g in ('FN','FP') else 0.45, label=GCN[g])
ax.set_xlabel(FEATURE_CN[fx]); ax.set_ylabel(FEATURE_CN[fy])
ax.set_title('(b) 关键特征平面'); ax.legend(fontsize=8)

ax = fig.add_subplot(gs[0,2], projection='polar')
ang = np.linspace(0, 2*np.pi, len(labels), endpoint=False).tolist(); ang += ang[:1]
for g in G:
    v = zmean.loc[g].tolist(); v += v[:1]
    ax.plot(ang, v, color=GC[g], lw=2, label=g)
    ax.fill(ang, v, color=GC[g], alpha=0.08)
ax.set_xticks(ang[:-1]); ax.set_xticklabels(labels, fontsize=9)
ax.set_title('(c) 四组特征画像', pad=18)
ax.legend(fontsize=8, loc='upper right', bbox_to_anchor=(1.3, 1.12))

ax = fig.add_subplot(gs[1, 0:2])
mat = zmean.copy()
mat.loc['FN-TP'] = zmean.loc['FN'] - zmean.loc['TP']
mat.loc['FP-TN'] = zmean.loc['FP'] - zmean.loc['TN']
lim = np.nanmax(np.abs(mat.values))
im = ax.imshow(mat.values, cmap='RdBu_r', vmin=-lim, vmax=lim, aspect='auto')
ax.grid(False)
```

```
for i in range(mat.shape[0]):
    for j in range(mat.shape[1]):
        ax.text(j, i, '%+.2f' % mat.values[i,j], ha='center', va='center',
                fontsize=9, color=CS['dark'])
ax.axhline(3.5, color=CS['dark'], lw=2)
ax.set_xticks(range(len(labels))); ax.set_xticklabels(labels)
ax.set_yticks(range(mat.shape[0]))
ax.set_yticklabels([GCN.get(r, r+' (差异)') for r in mat.index])
fig.colorbar(im, ax=ax, pad=0.01, fraction=0.03, label='z 分数')
ax.set_title('(d) 特征画像热力图：下两行 = 错误组相对正确组的偏移')

ax = fig.add_subplot(gs[1,2])
bins = np.linspace(0, 1, 26); bottom = np.zeros(len(bins)-1)
for g in G:
    h, _ = np.histogram(prob[grp==g], bins=bins)
    ax.bar(bins[:-1], h, width=np.diff(bins), bottom=bottom, align='edge',
           color=GC[g], edgecolor='white', label=g)
    bottom += h
ax.axvline(t, color=CS['dark'], ls='--', lw=2, label='阈值 %.2f' % t)
ax.set_xlabel('预测违约概率'); ax.set_ylabel('样本数')
ax.set_title('(e) 堆叠概率直方图'); ax.legend(fontsize=8)

plt.suptitle('误分类诊断（规模 / 空间 / 画像 / 概率位置）',
             fontsize=15, fontweight='bold')
plt.savefig('18_高级_误分类诊断.png', dpi=150, bbox_inches='tight')
```

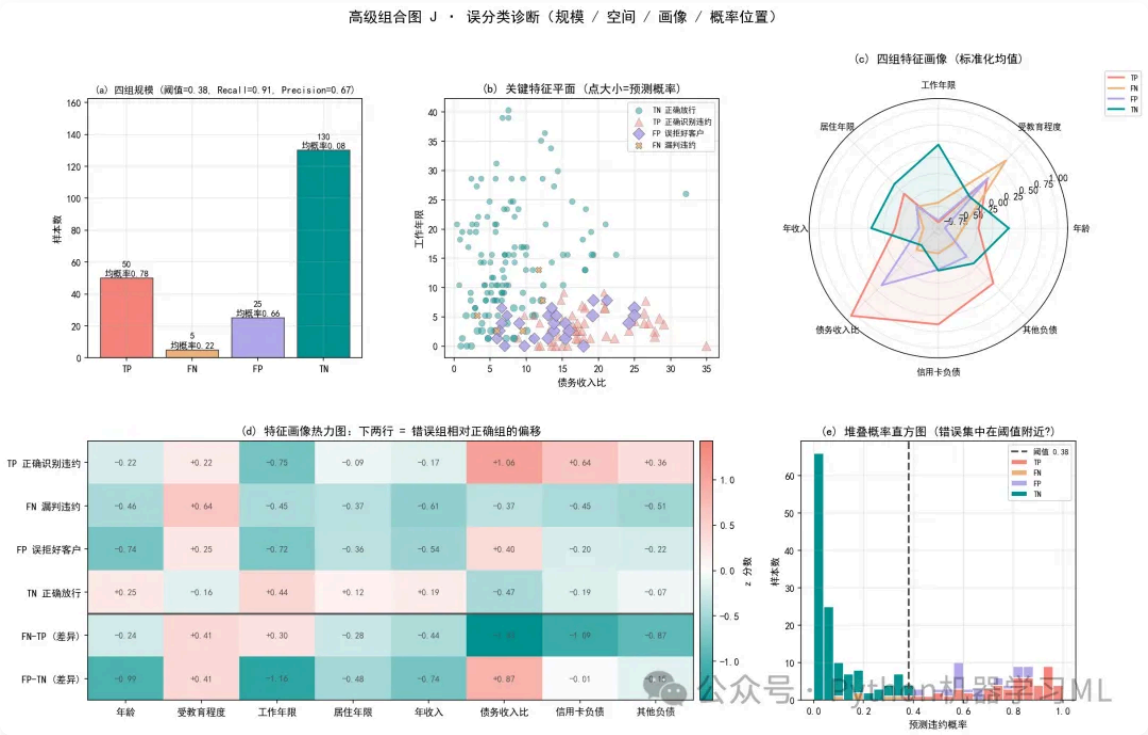


图 18 · 四组误分类人群画像

怎么读：

- **(a) 四组柱 + 均概率**
  ：FN 的均概率接近阈值 = 阈值再往下调一点就能救回部分违约。
- **(b) 平面散点**

：FN 的 X 点集中在哪个区域，就是模型的"知识盲区"。

- **(c) 雷达画像**
  ：FN 曲线接近 TN 而远离 TP，说明"漏判的违约客户确实长得像好客户"。
- **(d) 差异热力图**
  ：FN-TP 行里越红的列 = FN 相对 TP 偏高的特征，是误分类的"元凶"。
- **(e) 堆叠直方图**
  ：错误如果集中在阈值附近 → 可考虑设"灰区人工复审"。

总结

18 张图从数据到模型、从解释到决策完整走了一遍。真正要落地时，记住三件事：

- **任何挑参数、挑阈值都不碰测试集**
  OOB、交叉验证、OOF 这些机制都是为了这件事。
- **不平衡场景不要只看 accuracy**
  至少同时看 AUC / AP / F1，再加 Recall、Precision、KS、Lift。
- **概率要用来做决策，就先做校准**
  Brier 高、校准曲线弯曲，就用 isotonic 修正一遍再上线。

⚠️ **可复用但不是开箱即用**：代码里的随机种子、CV 折数、代价比例、阈值都应该按业务重新设定。文章里的默认值只是演示用。

📘 完整代码 + 数据请私信获取

关注公众号，不错过更新

✨ AI工具推荐 ✨

官网：https://artdraw.me

Python机器学习ML
常见数学模型、算法、机器学习、深度学习实战
121篇原创内容

公众号

Python_ml
中国大陆 重庆

公众号 · Python机器学习ML

历史推文：

机器学习——审稿人要求的统计检验、ROC曲线、置信区间、特征重要性……这一套决策树代码全部搞定

顶刊复现 | IF=10.6！基于可解释机器学习评估免疫炎症指标在非酒精性脂肪肝病（NAFLD）患者中预测冠心病（CHD）风险

多种机器学习模型+Stacking模型的SHAP可视化+二分类+SMOTE处理不平衡数据

13种机器学习模型+多分类+PCA+贝叶斯优化超参数+SHAP高级可视化

机器学习——线性回归、决策树、随机森林、LightGBM模型的SHAP和LIME可解释性分析对比详解完整示例

机器学习——因果推断方法的DeepIV和因果森林双重机器学习（CausalForestDML）示例

论文复现——肺癌数据高级模型比较与shap可视化分析代码解析

论文复现——肺癌预测数据分析与逻辑回归、朴　素贝叶斯、支持向量机、随机森林、K近邻、XGBoost、深度神经网络模型评估代码解析

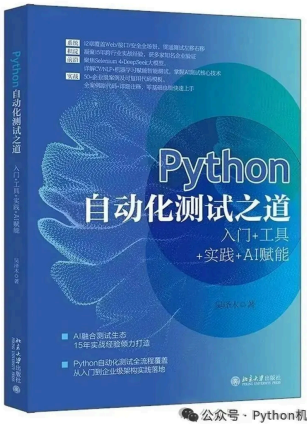
机器学习——材料力学XGBoost 分类，SHAP可视化分析、SMOTE解决不平衡、PCA降维、统计分析完整代码解析

机器学习——SHAP可解释分析、EDA、逻辑回归、支持向量机（SVM）电动车数据集Python完整代码

赠书活动

支持知识分享，畅享学习乐趣！特别感谢**北京大学出版社**对本次赠书活动的鼎力支持！即日起，只需**点赞、评论**此文章，作者将随机抽取一位幸运儿，**免费包邮赠送**北京大学出版社提供的《Python自动化测试之道：入门+工具+实践+AI赋能》这本精彩书籍 📚！

💡 赶快参与，说不定你就是那位幸运读者哦！



公众号 · Python机器学习ML

随机森林算法 · 目录

上一篇 · 机器学习——需求预测+PCA+随机森林算法+shap可解释性分析+多模型性能对比

作者提示: 个人观点，仅供参考